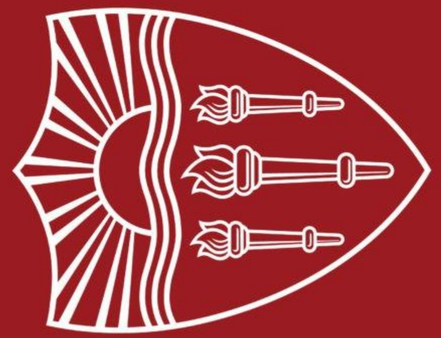




Lecture 16:
LLM prompting, scaling laws & Issues

Instructor: Xiang Ren

USC CSCI 544 Applied NLP

Fall 2026

Some slides adapted from Tao Yu, Mohit Iyyer, Pat Virtue & Matt Gormley

Announcement

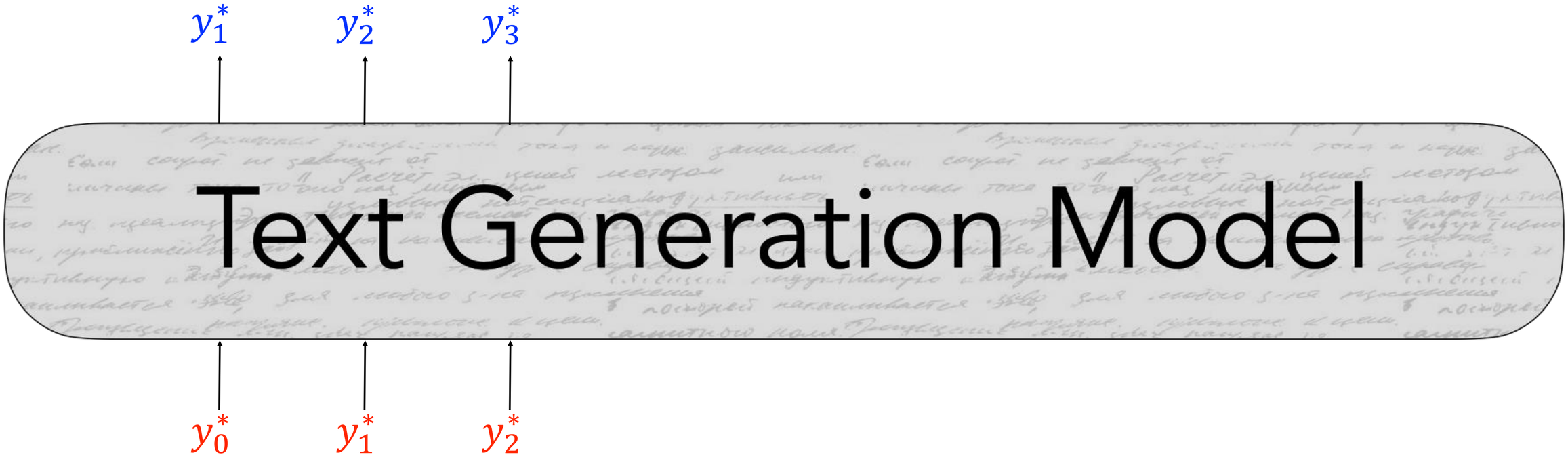
- Midterm presentation grades and feedback released.
- Peer review feedback
- Midterm report grade and feedback in ~two weeks.
- HW2 due Nov 10

Recap:
NL Generation & Evaluation

Maximum Likelihood Training

- Trained to generate the next word y_t^* given a set of preceding words $\{y^*\}_{<t}$

$$L = -(\log P(y_1^*|y_0^*) + \log P(y_2^*|y_0^*, y_1^*) + \log P(y_3^*|y_0^*, y_1^*, y_2^*))$$

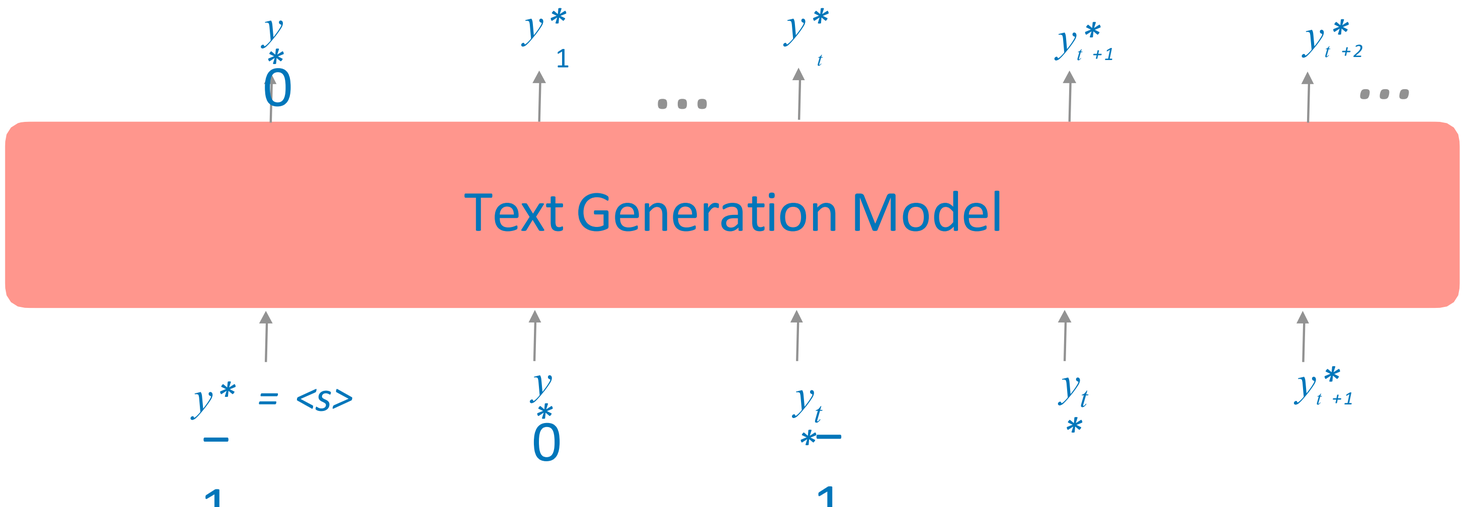


Language Generation: Training

- Trained one token at a time to maximize the probability of the next token y_t^* given preceding words $y_{<t}^*$

$$\mathscr{L} = -\sum_{t=1}^{T} \log P(y_t | y_{<t}) = -\sum_{t=1}^{T} \log \frac{\exp(S_{y_t|y_{<t}})}{\sum_{v \in V} \exp(S_{v|y_{<t}})}$$

- Classification task at each time step trying to predict the actual word y_t^* in the training data
- "Teacher forcing" (reset at each time step to the ground truth)



Language Generation: Inference

- At each time step *t*, our model computes a vector of scores for each token in our vocabulary, $S \in \mathbb{R}^{V}$

$$S = f(\{y_{<t}\}; \theta)$$ $f(\cdot;\theta)$ **is your model**

- Then, we compute a probability distribution P over $w \in V$ using these scores:

$$P(y_t = w|\{y_{<t}\}) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

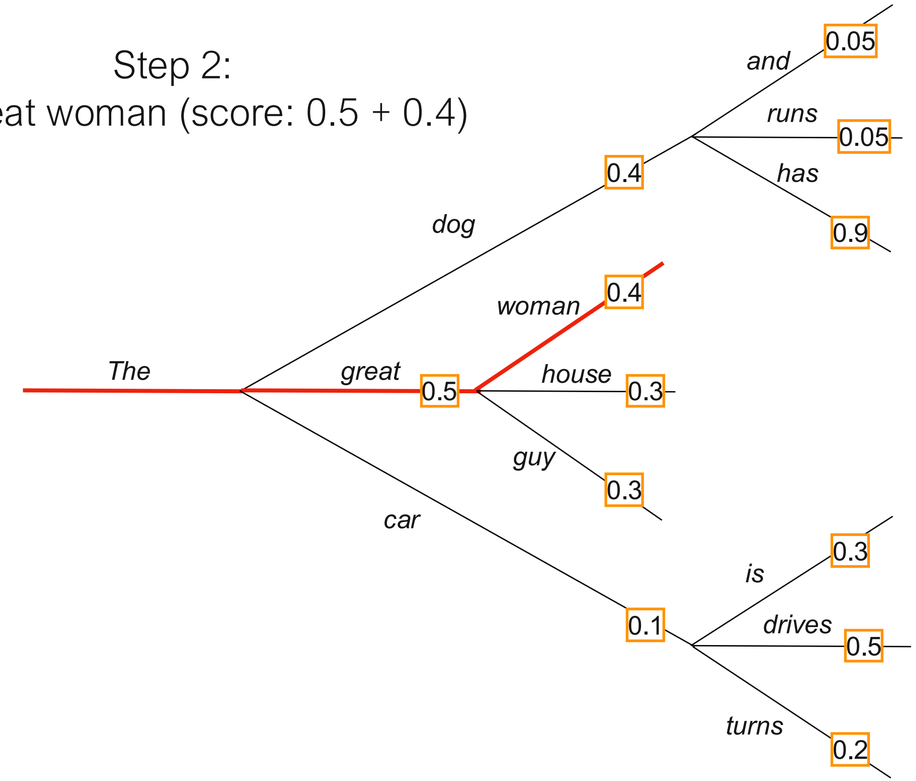
- Our **decoding** algorithm defines a function to select a token from this distribution:

$$\hat{y}_t = g(P(y_t|\{y_{<t}\}))$$ $g(\cdot)$ **is your decoding algorithm**

Note: *we decode token by token from LLMs after they are trained (during inference)*

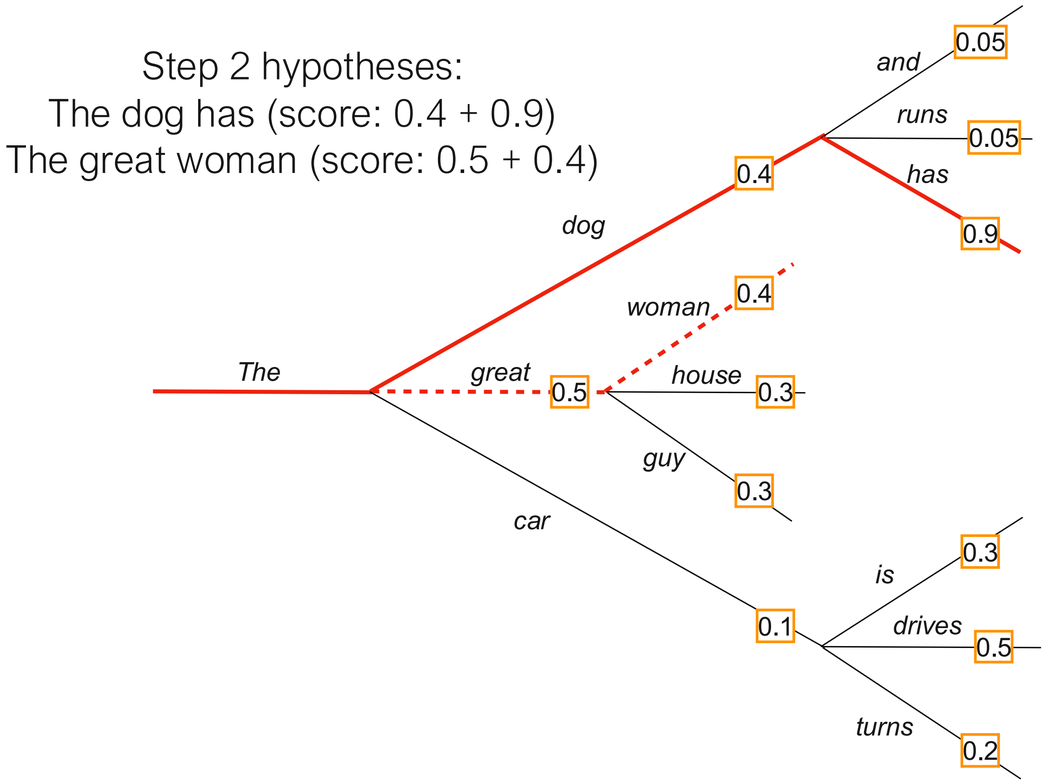
Greedy Decoding vs. Beam Search

- **Greedy Decoding**
  - Choose the "currently best" token at each time step



Greedy Decoding vs. Beam Search

- **Beam Search (in this example, *beam_width* = 2)**
  - At each step, retain 2 hypotheses with the highest probability



Maximization Based Decoding

- Either greedy or beam search
- Beam search can be more effective with large beam width, but also more expensive
- Another key issue:

Generation can be bland or repetitive

(also called degenerate)

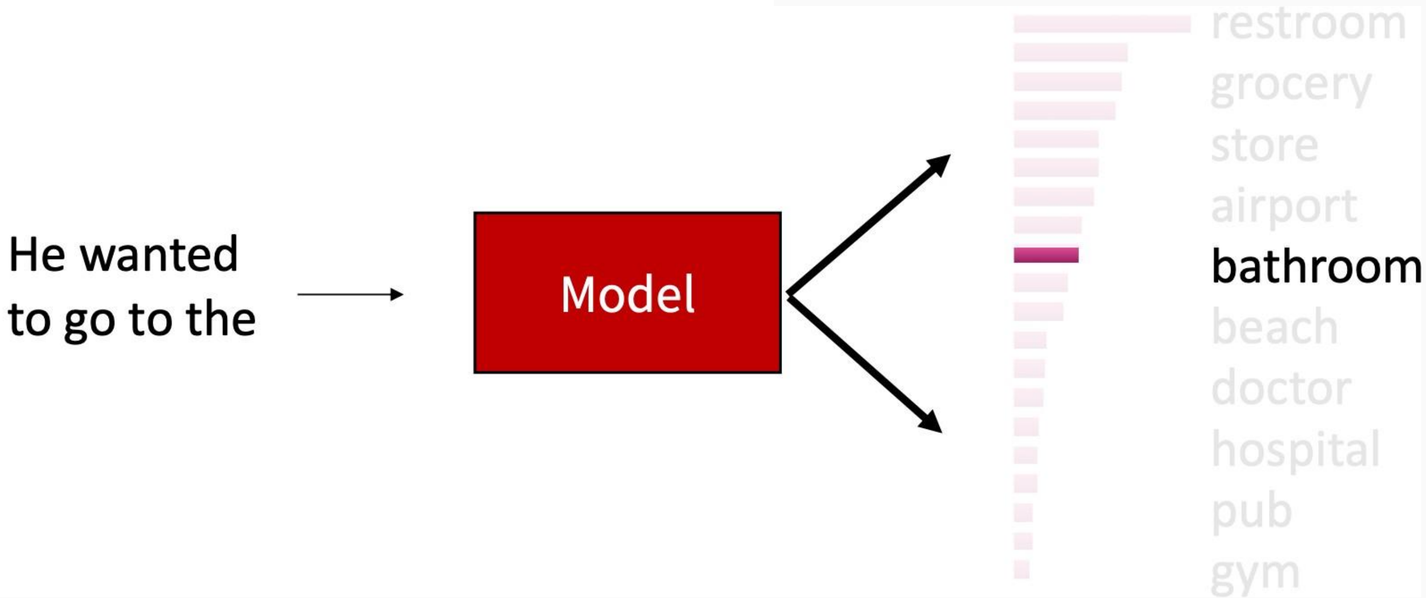
Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Continuation: The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the **Universidad Nacional Autónoma de México (UNAM)** and **the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México…**

Holtzmann et al., 2020

Solution: Don't Maximize, Pick a Sample

- Sample a token from the distribution of tokens.
- But this is not a random sample, it is a sample for the learned model distribution
  - Respects the probabilities, without going just for the maximum probability option
  - Or else, you would get something meaningless
  - Many good options which are not the maximum probability!



Top-*K* Sampling: Issues

Top-K sampling can cut off too quickly

Top-K sampling can also cut off too slowly!

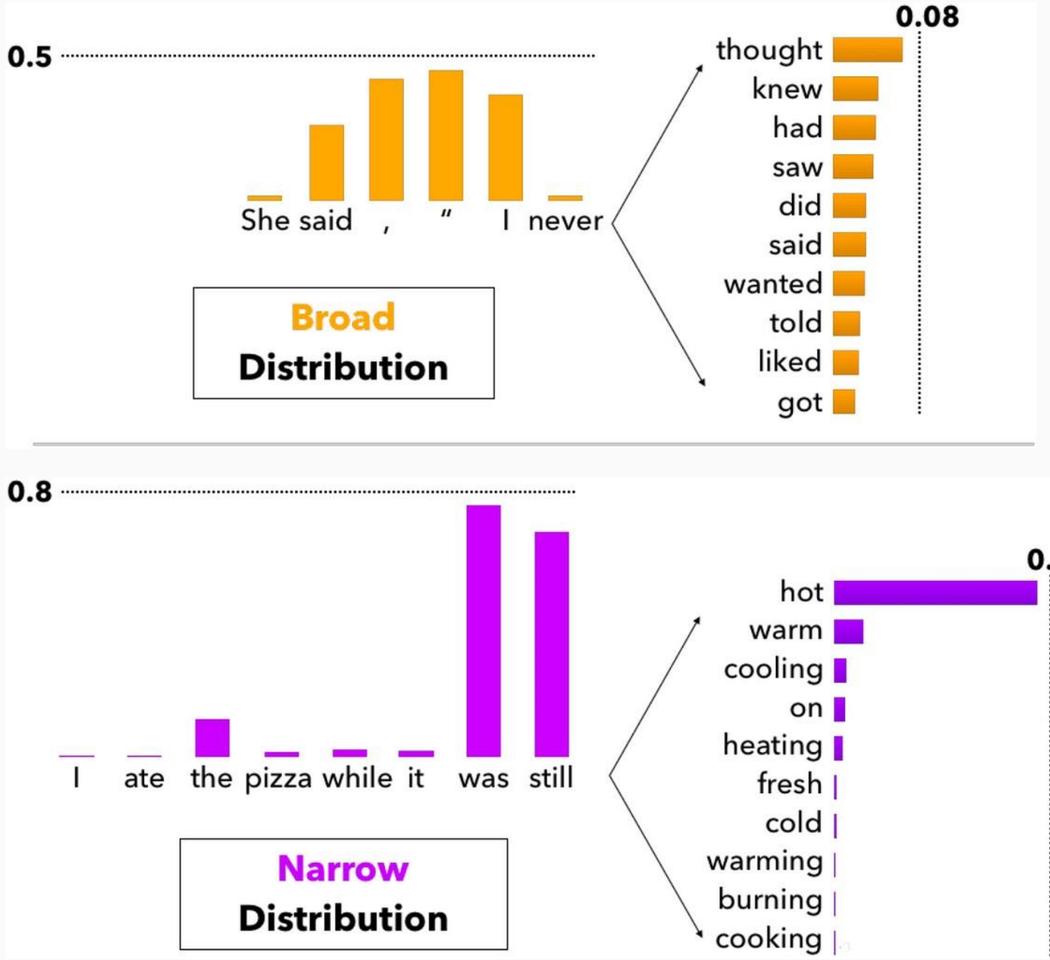
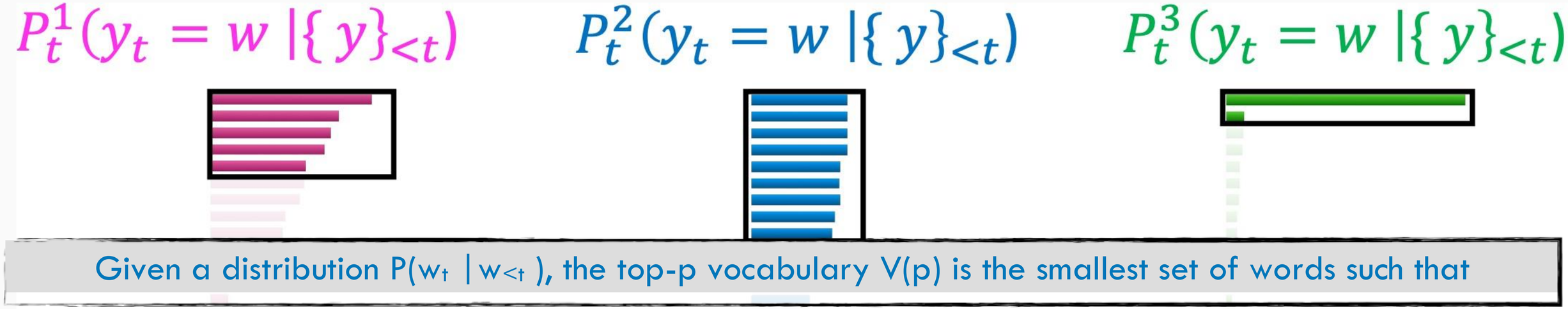


Image Source: Holtzmann et al., 2019

Nucleus (Top-*P*) Sampling

- Solution: Top-*P* sampling
  - Sample from all tokens in the top *P* cumulative probability mass (i.e., where mass is concentrated)
  - Varies *K* depending on the uniformity of P_t



Given a distribution $P(w_t \mid w_{<t})$, the top-p vocabulary V(p) is the smallest set of words such that

$$\sum_{w \in V(p)} P(w|\mathbf{w}_{<t}) \geq p$$

Holtzman et al., ICLR 2020

Scaling randomness: Softmax temperature

- You can apply **temperature hyperparameter** τ to the softmax to rebalance P_t:

$$P_t(y_t = w|\{y_{<t}\}) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

- Raise the temperature $\tau > 1$: P_t becomes more uniform
  - More diverse output (probability is spread across vocabulary)
- Lower the temperature $\tau < 1$: P_t becomes more spiky
  - Less diverse output (probability concentrated to the top tokens)

NOTE: Temperature is a hyperparameter for decoding algorithm, not an algorithm itself! It can be applied for both beam search and sampling methods.

Speeding-up generation: Speculative Sampling

- Problem: Generating with a large LM takes a long time

- Intuition: Not all tokens are equally hard to generate!

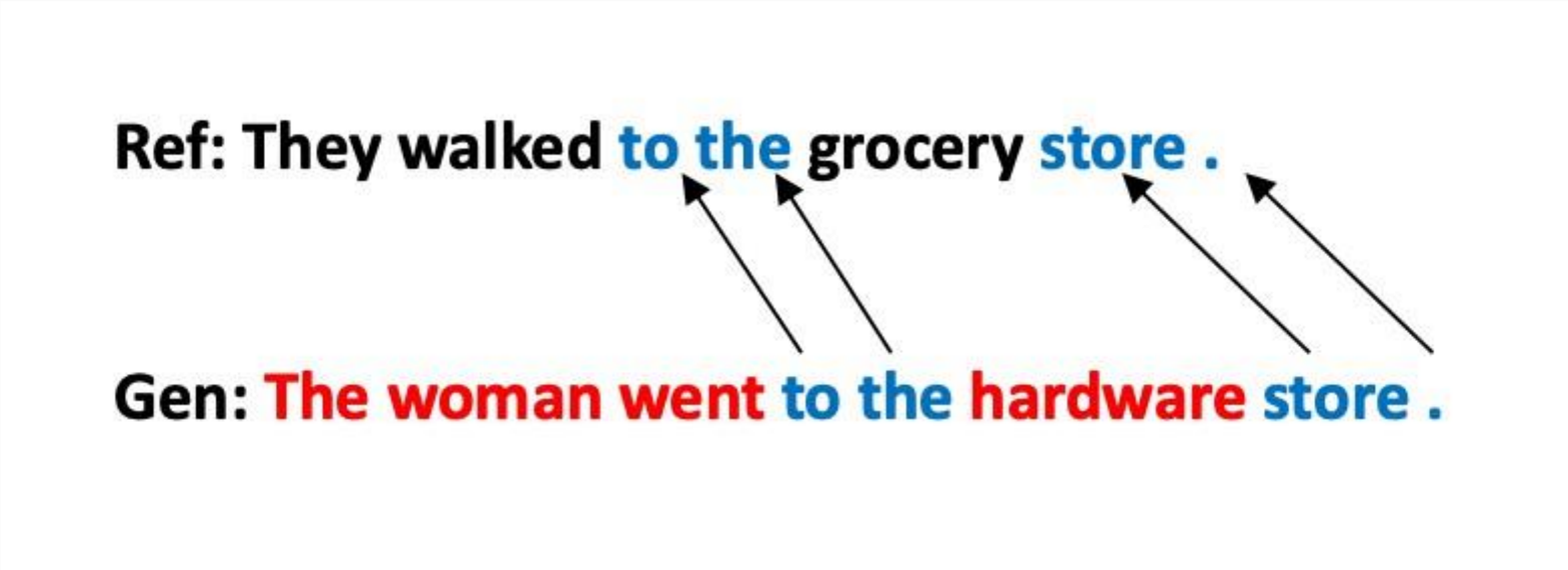


- **Idea**: Use a generation from small LM to assist large LM generation

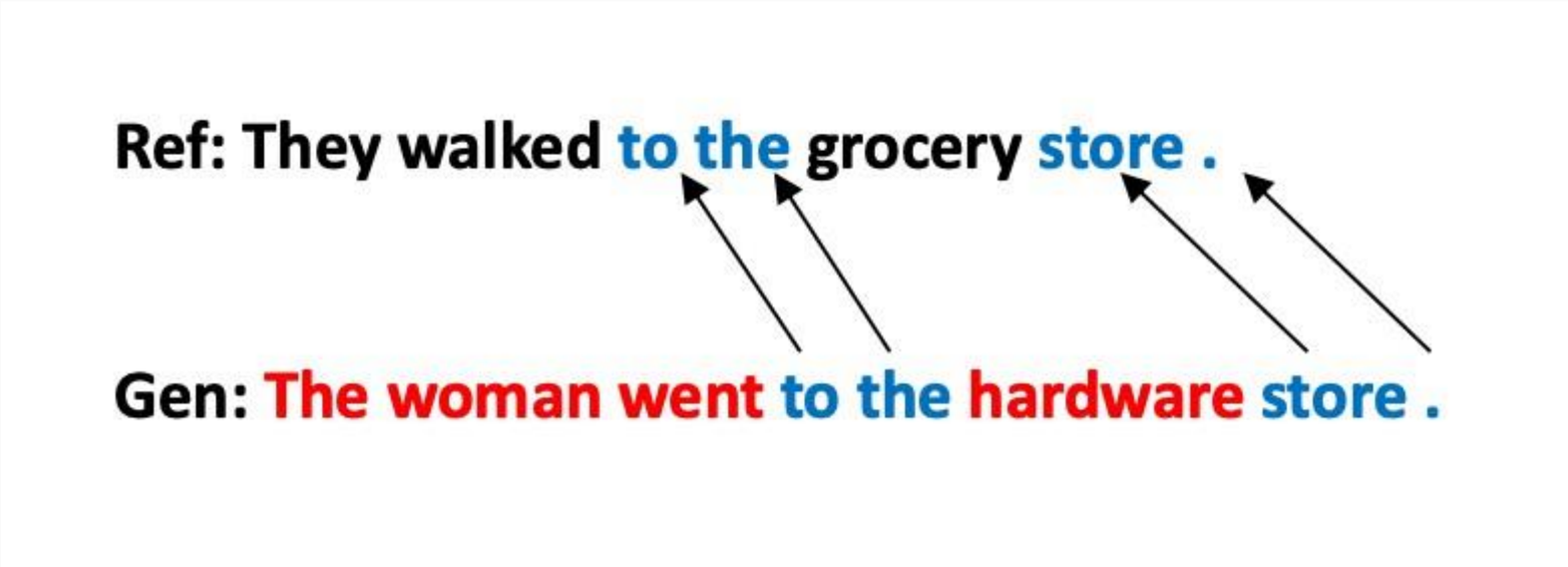
** Same idea independently proposed from DeepMind and Google - see Chen et al., 2023; Leviathan et al., 2023*

Evaluation Strategies

- With Reference
  - Lexical Matching
  - Semantic Matching
- Without Reference
  - Perplexity
  - Model-Based Metrics
  - Advanced: Distributional Matching
  - Even simpler and least reliable: Auto Evaluation
- Human Evaluation



Reference-Based Metrics

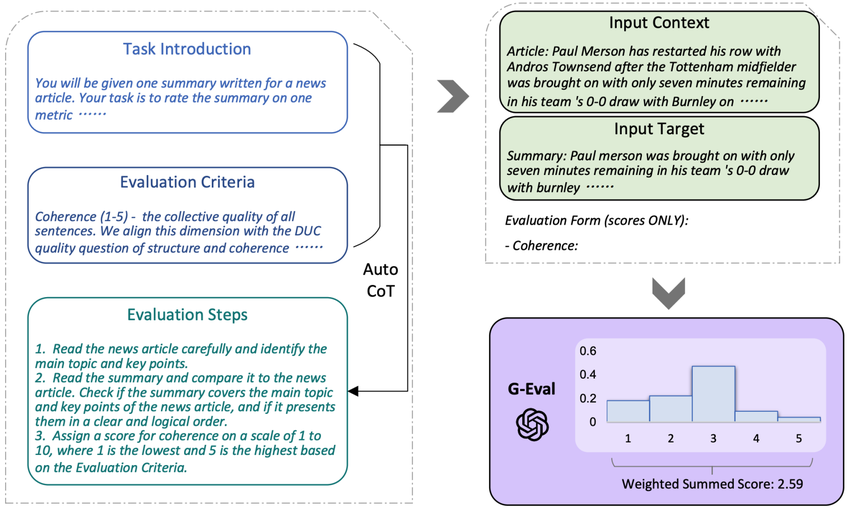


- Only possible for close-ended generation tasks
- Compute a score that indicates the lexical similarity between generated and gold- standard (human-written) text
- Fast and efficient and widely used
- n-gram overlap metrics (e.g., BLEU, ROUGE, etc.)

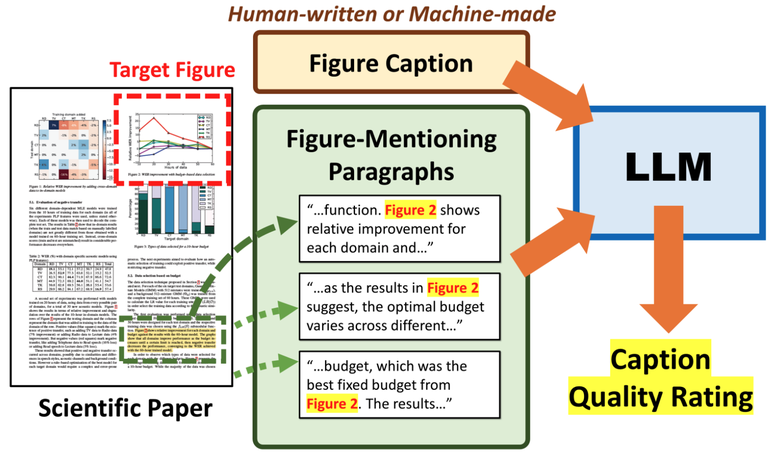
Model-based metrics: LLM as evaluator

- Directly prompt LLM (GPT-4) to evaluate generated text.
  - Can be customized with evaluation criteria
  - (Often) better correlation with human evaluators than task-specific metrics (e.g. ROUGE)
  - (Often) is cheaper than human evaluation

- Limitations
  - Brittleness: LLM evaluation can significantly vary when **given different prompts**!
  - Potential self-bias - LLMs may prefer what LLMs have generated...



Liu et al. 2023



Hsu et al. EMNLP Findings, 2023

Evaluation: Takeaways

- *Content-overlap metrics* provide a good starting point for evaluating the generation quality, but they're not good enough on their own

- *Model-based metrics* can be more correlated with human judgment, but often are not interpretable

- Human judgments are critical
  - But humans are inconsistent!

- In many cases, the best judge of output quality is **YOU**!
  - **Look at the actual generations - don't just rely on numbers.**
  - **Publicly release large samples of outputs from your system!**

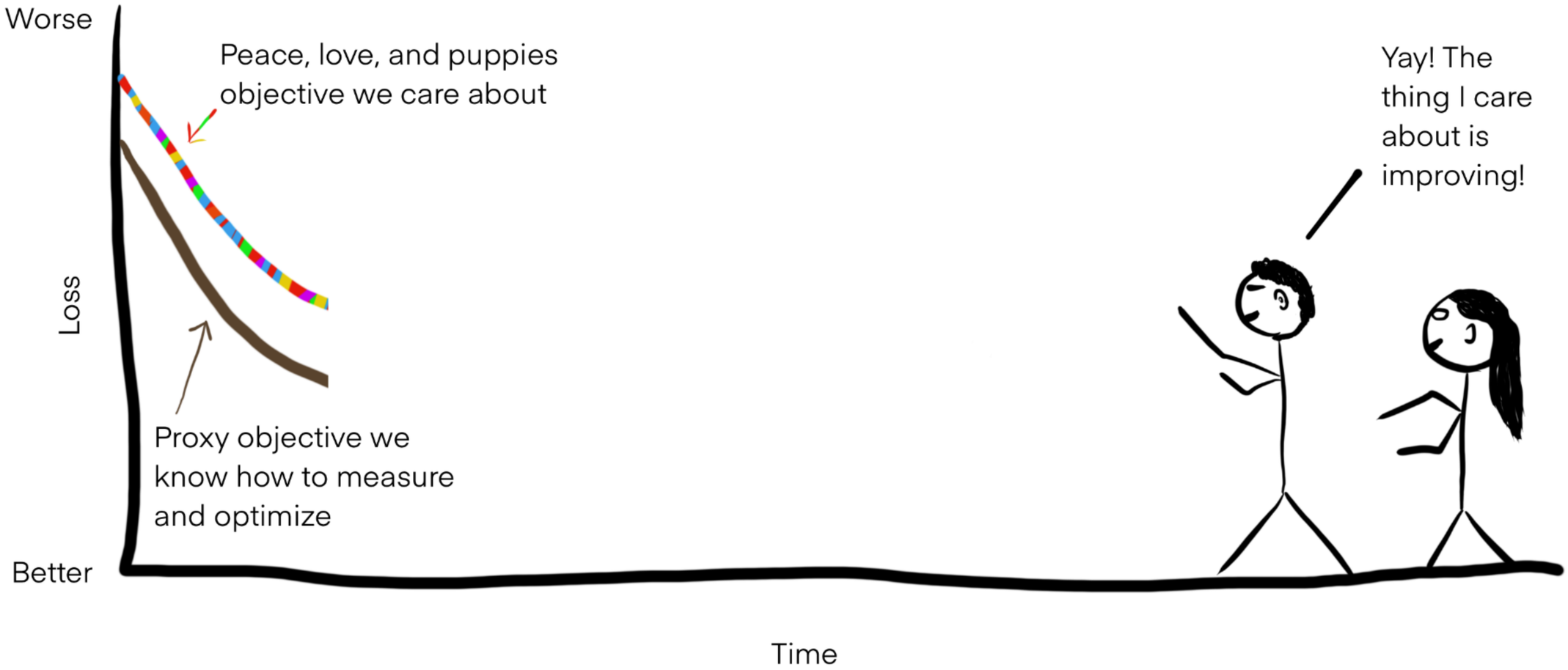
Everything in evaluation boils down to…

- *How should we evaluate: Evaluation Metrics*
- *What should we evaluate: Benchmarks and Tasks*

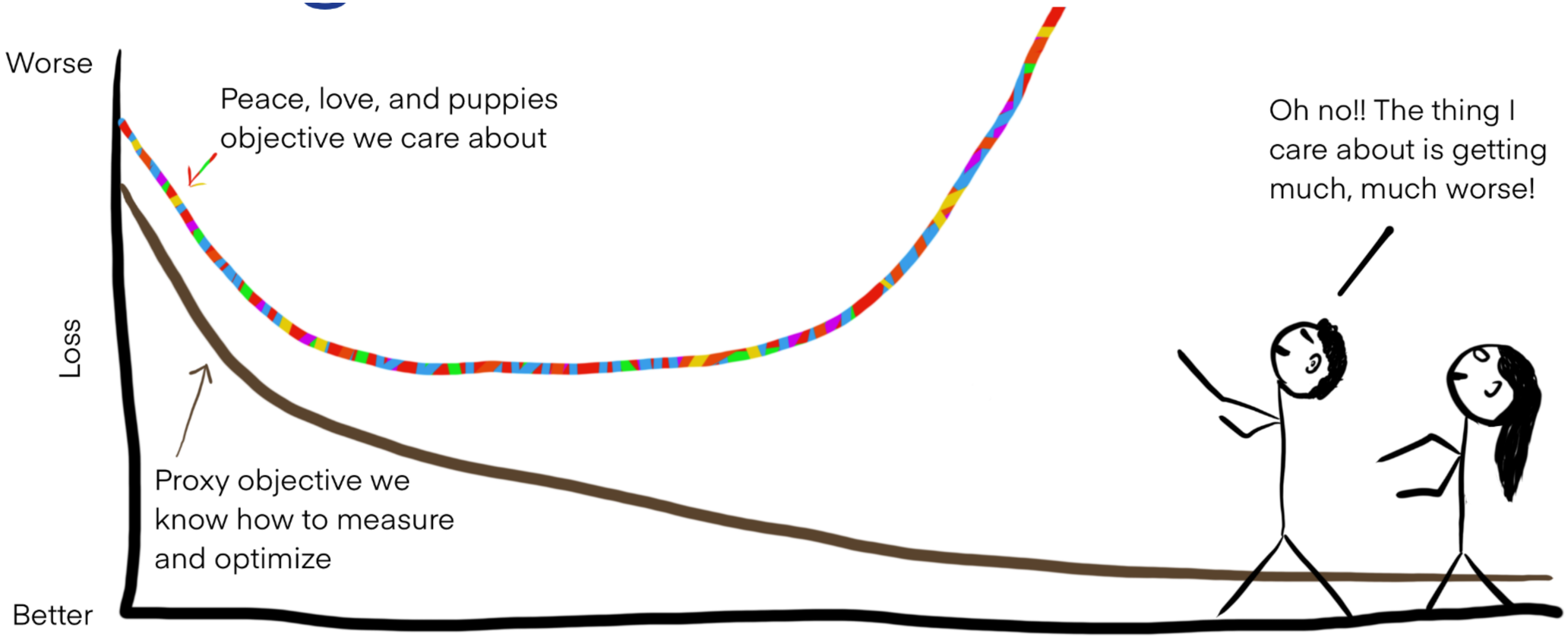
Goodhart’s laws: “when a measure becomes a target, it ceases to be a good measure"



Goodhart's laws: "when a measure becomes a target, it ceases to be a good measure"



Goodhart's laws: "when a measure becomes a target, it ceases to be a good measure"



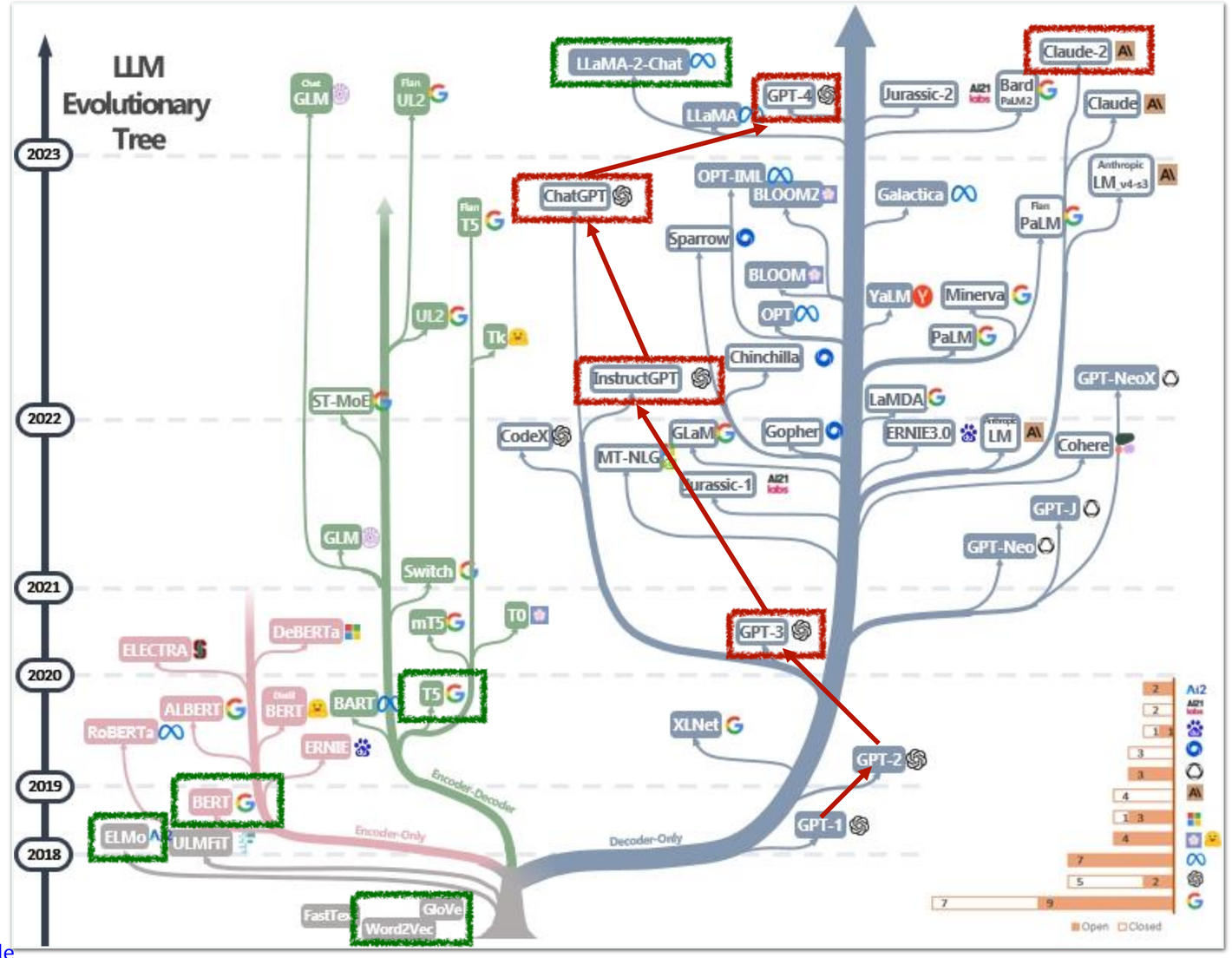
In-context Learning

Evolution tree of pretrained LMs

~200 billion

Model size
(# of parameters)
~1000 times larger

~300 million

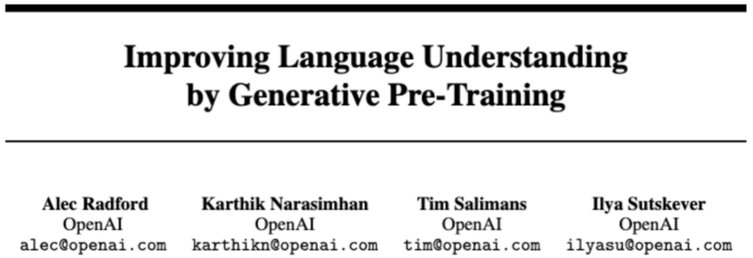


Open-sourced

Close-sourced

https://github.com/Mooler0410/LLMsPracticalGuide
https://mistral.ai/news/mistral-large/

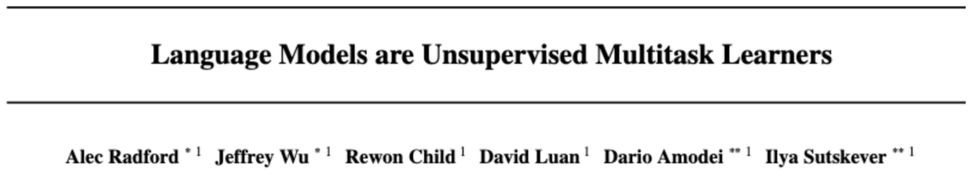
The GPT Family



Improving Language Understanding by Generative Pre-Training

Alec Radford OpenAI alec@openai.com
Karthik Narasimhan OpenAI karthikn@openai.com
Tim Salimans OpenAI tim@openai.com
Ilya Sutskever OpenAI ilyasu@openai.com

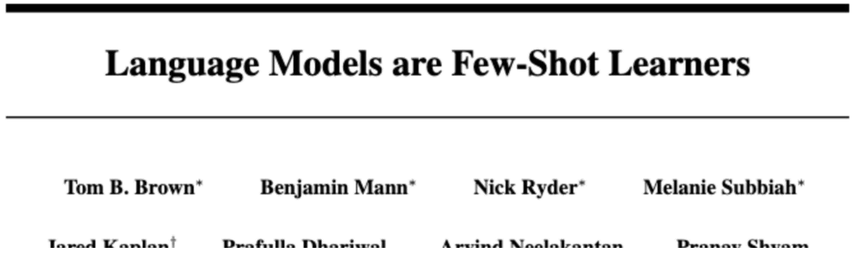
GPT (2018), 117 million parameters



Language Models are Unsupervised Multitask Learners

Alec Radford [*1] **Jeffrey Wu** [*1] **Rewon Child** [1] **David Luan** [1] **Dario Amodei** [**1] **Ilya Sutskever** [**1]

GPT-2 (2019), 1.5 billion parameters



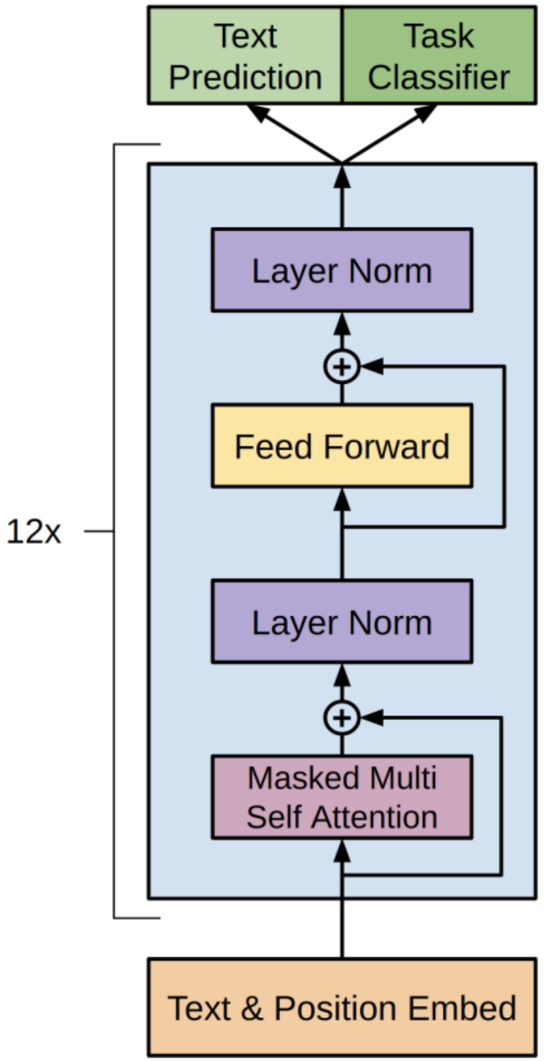
Language Models are Few-Shot Learners

Tom B. Brown* **Benjamin Mann*** **Nick Ryder*** **Melanie Subbiah***
Jared Kaplan† **Prafulla Dhariwal** **Arvind Neelakantan** **Pranav Shyam**

GPT-3 (2020), 175 billion parameters

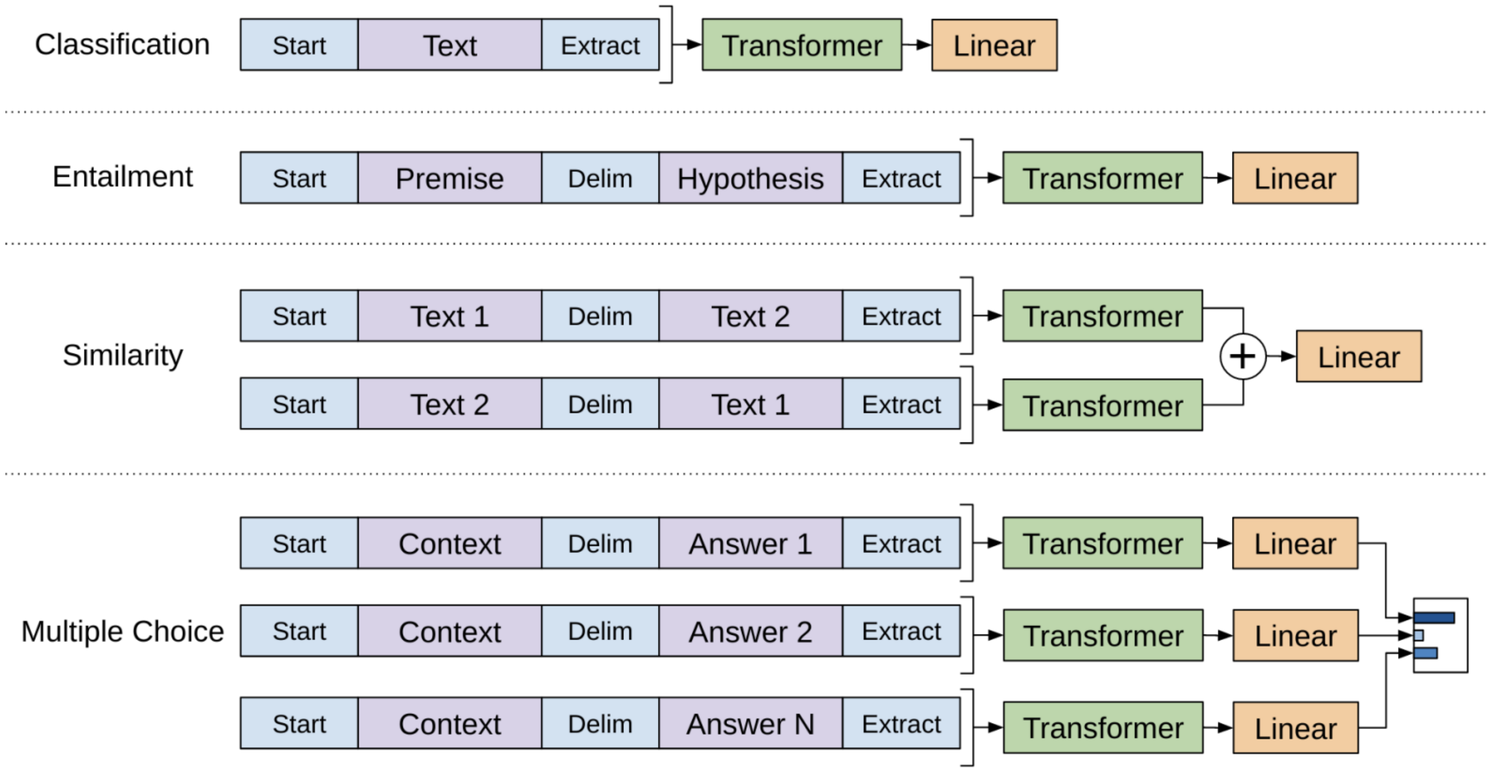
NeurIPS 2020 best paper

The 1st GPT model (GPT-1)



Pretrained on the BooksCorpus

Also shows results on fine-tuning for end tasks, where inputs and outputs are converted to text



GPT-2 is identical to GPT-1, but:

- Layer norm moved to the input of each sub-block
- Vocabulary extended to 50,257 tokens and context size increased from 512 to 1024
- Trained on 8 million docs from the web (Common Crawl), minus Wikipedia

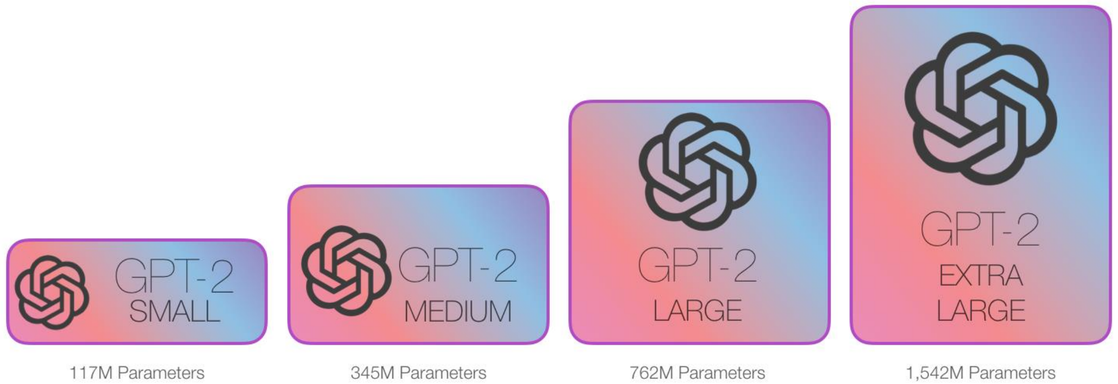
From GPT1 to GPT-2

- All **decoder-only Transformer-based language models**
- Model size ↑, training corpora ↑

GPT-2



Context size = 1024



.. trained on 40Gb of Internet text ..

(Radford et al., 2019): Language Models are Unsupervised Multitask Learners

GPT2: Some Results

Language Models are Unsupervised Multitask Learners

	LAMBADA (PPL)	LAMBADA (ACC)	CBT-CN (ACC)	CBT-NE (ACC)	WikiText2 (PPL)	PTB (PPL)	enwik8 (BPB)	text8 (BPC)	WikiText103 (PPL)	1BW (PPL)
SOTA	99.8	56.25	85.7	82.3	39.14	46.54	0.99	1.08	18.3	**21.8**
117M	**35.13**	45.99	**87.65**	**83.4**	**29.41**	65.85	1.16	1.17	37.50	75.20
345M	**15.60**	55.48	**92.35**	**87.1**	**22.76**	47.33	1.01	**1.06**	26.37	55.72
762M	**10.87**	**60.12**	**93.45**	**88.0**	**19.93**	**40.31**	**0.97**	**1.02**	22.05	44.575
1542M	**8.63**	**63.24**	**93.30**	**89.05**	**18.34**	**35.76**	**0.93**	**0.98**	**17.48**	42.16

Table 3. Zero-shot results on many datasets. No training or fine-tuning was performed for any of these results. PTB and WikiText-2 results are from (Gong et al., 2018). Other language model results are from (Dai et al., 2019).

(Radford et al., 2019): Language Models are Unsupervised Multitask Learners

GPT-3: More everything



GPT-3: A Very Large GPT Model (2020)

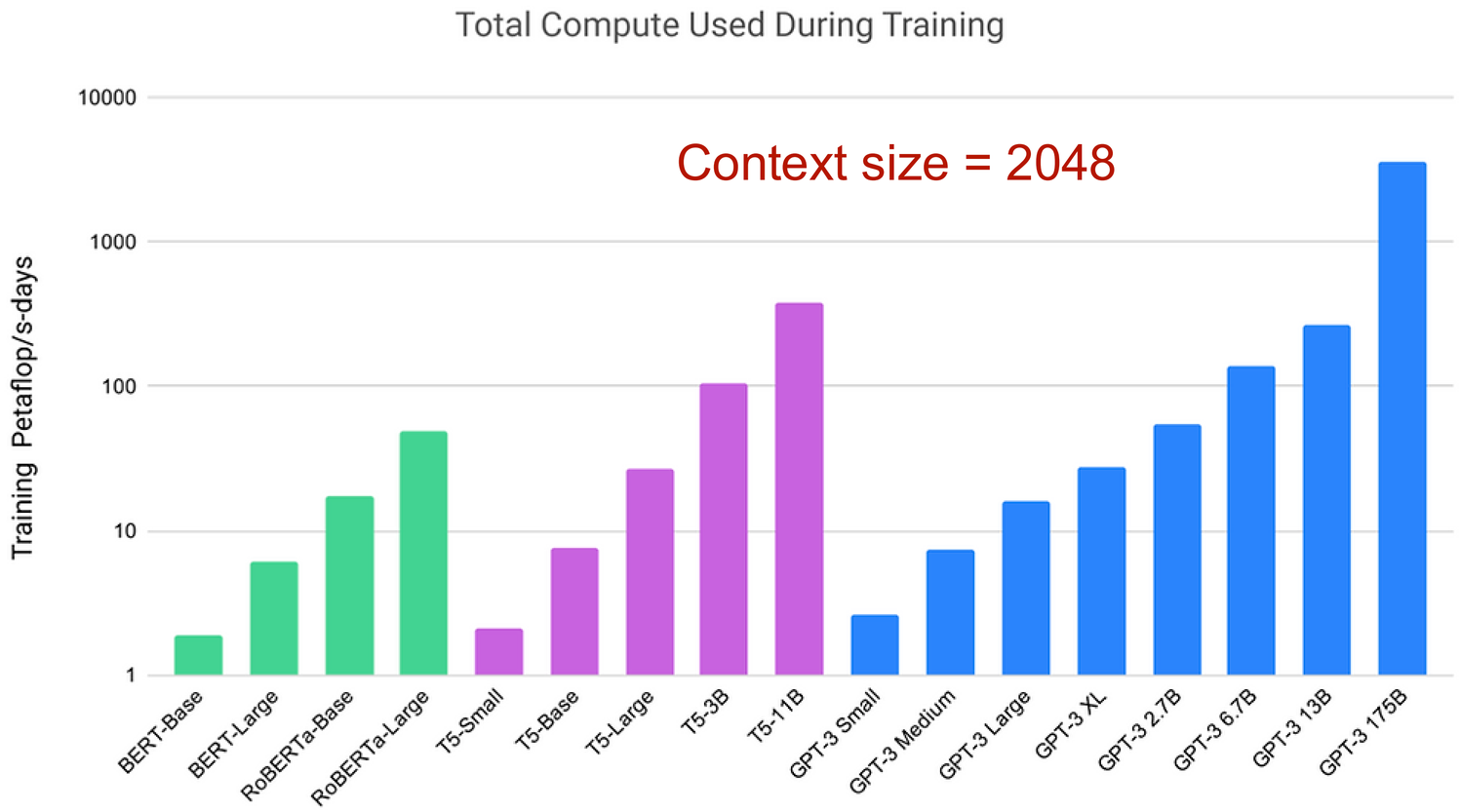
- More layers & parameters
- Bigger dataset
- Longer training
- Larger embedding/hidden dimension
- Larger context window



(Brown et al., 2020): Language Models are Few-Shot Learners

GPT-3: language models are few-shot learners

- GPT-2 → GPT-3: 1.5B → 175B (# of parameters), ~14B → 300B (# of tokens)



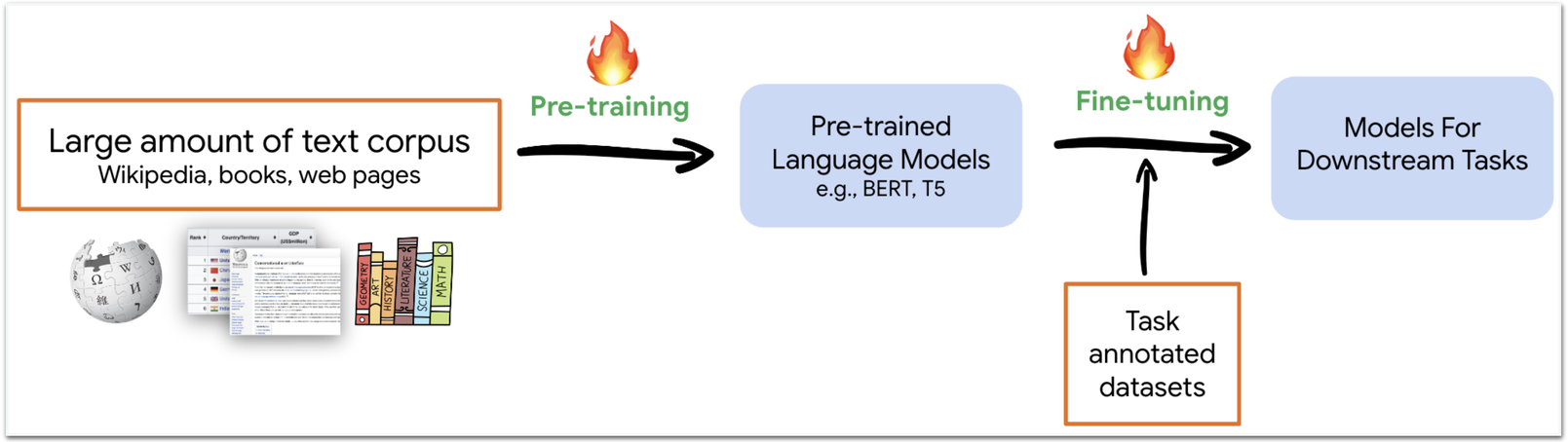
(Brown et al., 2020): Language Models are Few-Shot Learners

Training computation is measured using floating-point operations or "FLOP".

One FLOP represents a single arithmetic operation involving floating-point numbers, such as addition, subtraction, multiplication, or division.

Before GPT3: Modern learning paradigm

- **Pre-training + supervised training/fine-tuning**
  - First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.
  - Then train the pretrained Transformer for a specific task using supervised learning. This is called **finetuning**.



Paradigm shift since GPT-3

- Before GPT-3, **Pre-training + supervised training/fine-tuning** is the default way of doing learning in models like BERT/T5/GPT-2

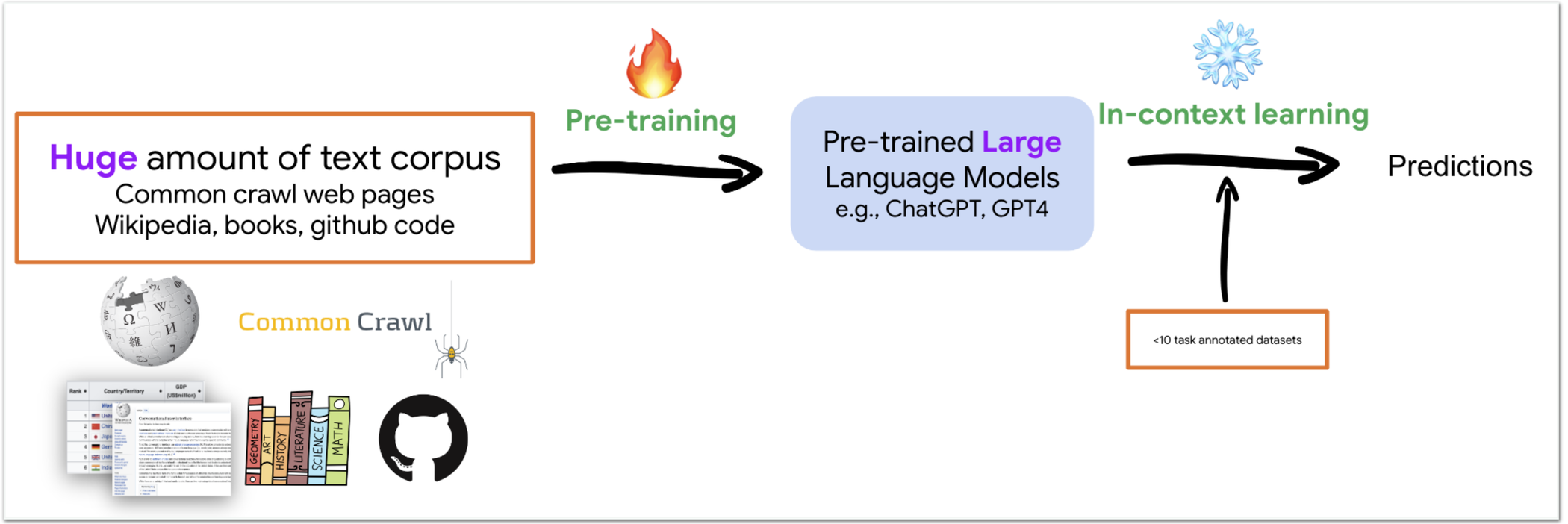
- Fine-tuning requires computing the gradient and applying a parameter update on every example (or every K examples in a mini-batch)

- However, this is very expensive for the 175B GPT-3 model



Latest learning paradigm shift since GPT-3

- **Pre-training + prompting/in-context learning (no training this step)**
  - **Directly use** the pretrained large Transformer (**no further finetuning/training**) for any different task given only a natural language description of the task or a few task (x, y) examples. This is called **prompting/in-context learning**.

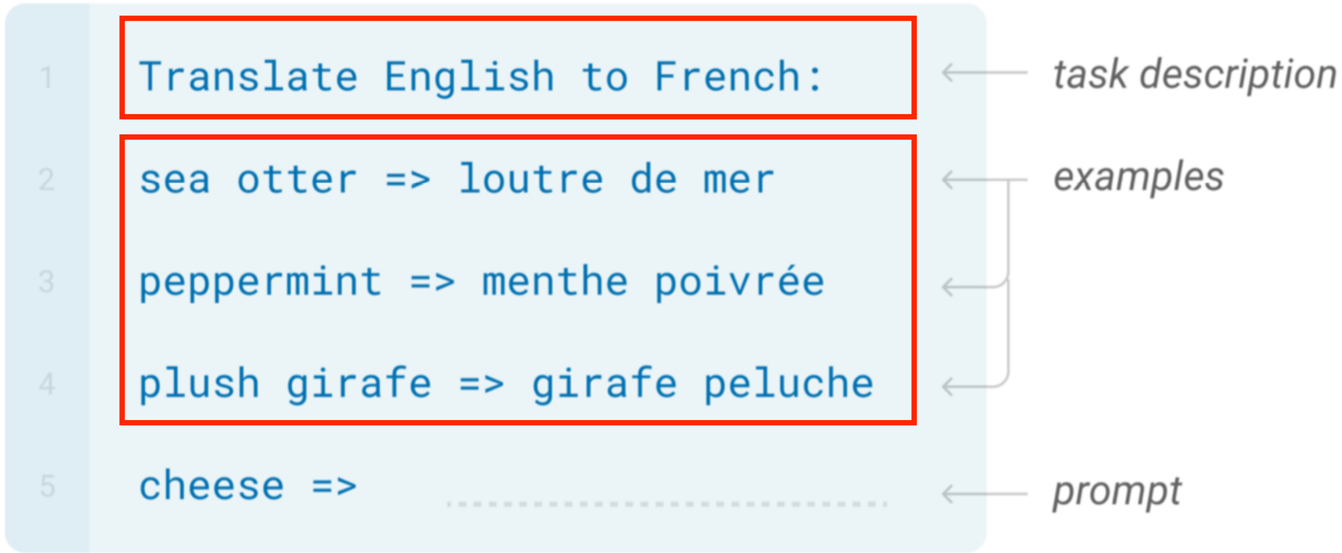


GPT-3: few-shot in-context learning

- GPT-3 proposes an alternative: **in-context learning**

Few-shot

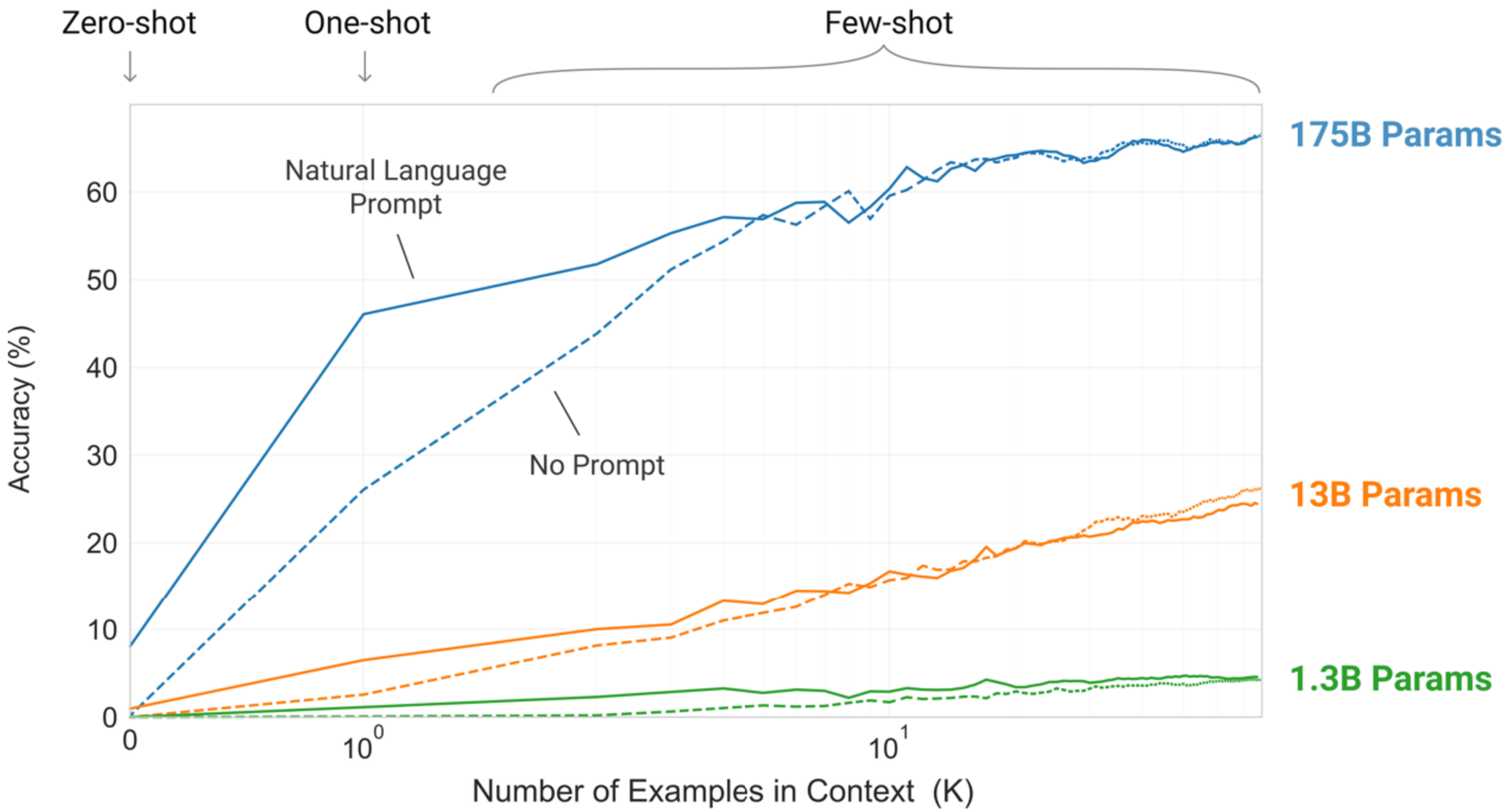
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



- This is just a forward pass, **no gradient update at all**!
- You only need to feed a small number of examples (e.g., 4-32)

(On the other hand, you can't feed many examples at once too as it is bounded by context size)

GPT-3's scaling laws in performance



(Brown et al., 2020): Language Models are Few-Shot Learners

Chain-of-thought (CoT) prompting

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

(Wei et al., 2022): Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Why in-context learning with LLMs?

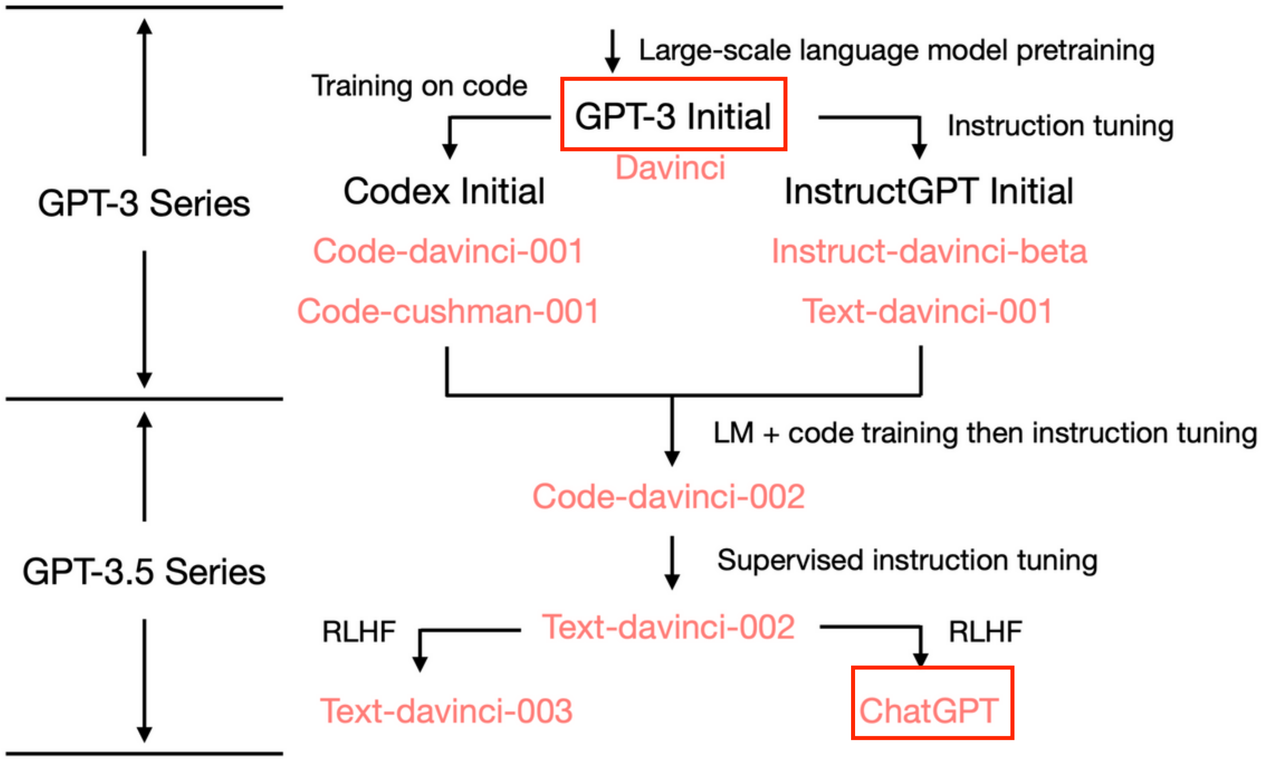
- Amazing zero/few-shot performance
  - Save a lot of annotation! 🎉
- Easy to use without training
  - Just talk to them! 👍
- One model for many NLP applications 😄
  - No need to annotate and fine-tune for different tasks

But, again, they are sensitive to prompts! Need to design a good prompt or train a good example retriever! 😂

What happened after GPT-3?

(Is model size ↑, training corpora ↑, compute ↑, the only way to go?)

How was ChatGPT developed?

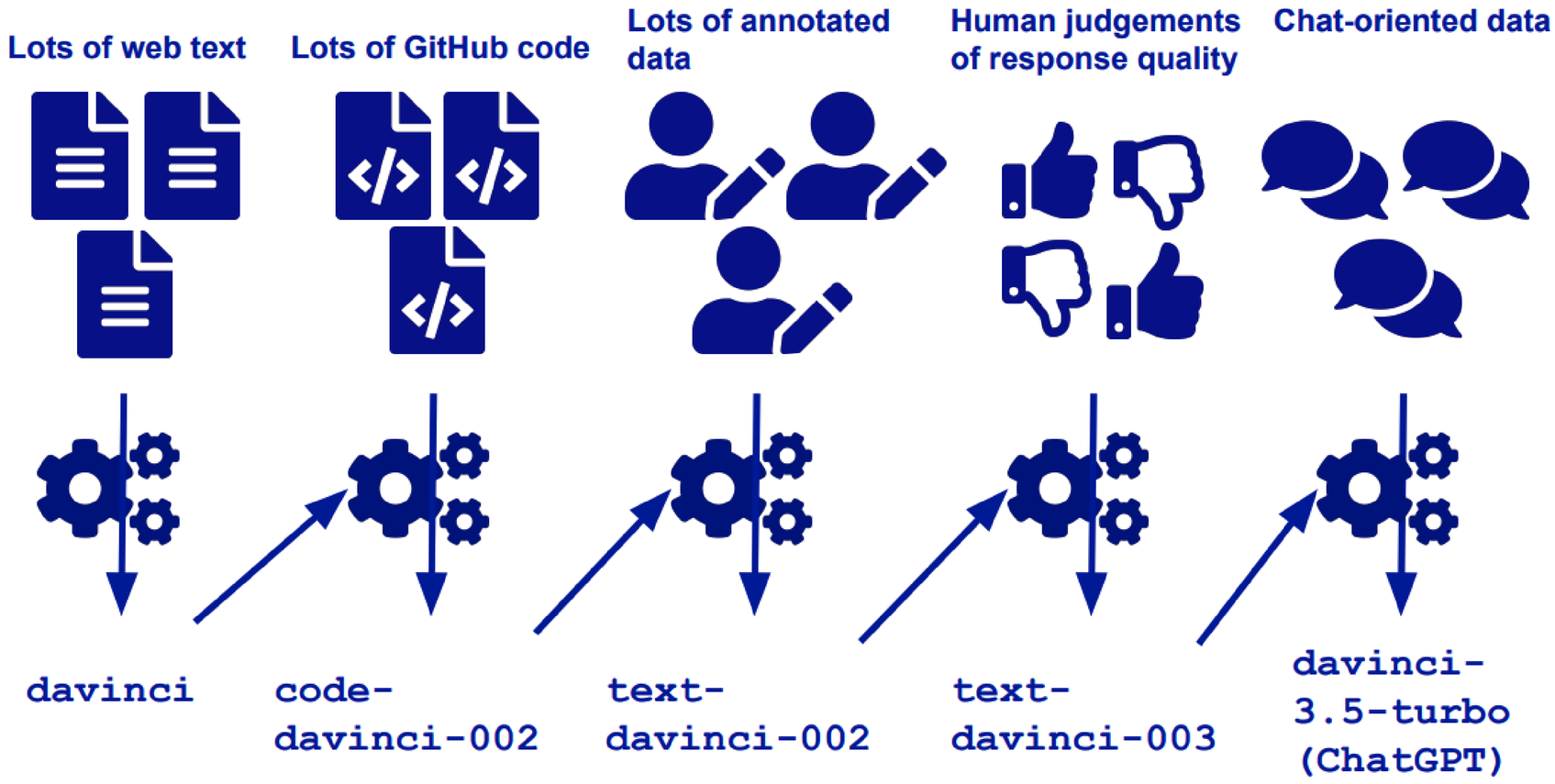


What's new?

- Training on code
- **Supervised instruction tuning**
- **RLHF = Reinforcement learning from human feedback**

https://yaofu.notion.site/How-does-GPT-Obtain-its-Ability-Tracing-Emergent-Abilities-of-Language-Models-to-their-Sources-b9a57ac0fcf74f30a1ab9e3e36fa1dc1

How was ChatGPT developed?



(Slide credit: Graham Neubig)

InstructGPT: Supervised instruction tuning + RLHF

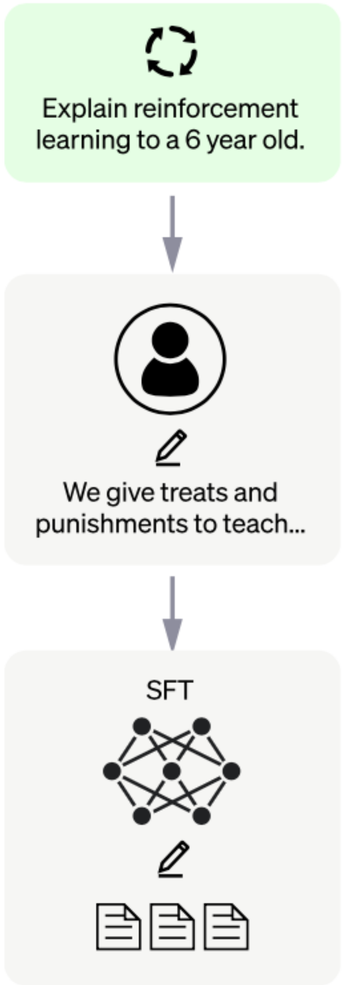
Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



(Ouyang et al., 2022): Training language models to follow instructions with human feedback

Supervised instruction tuning

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Number of Prompts

SFT Data		
split	source	size
train	labeler	11,295
train	customer	1,430
valid	labeler	1,550
valid	customer	103

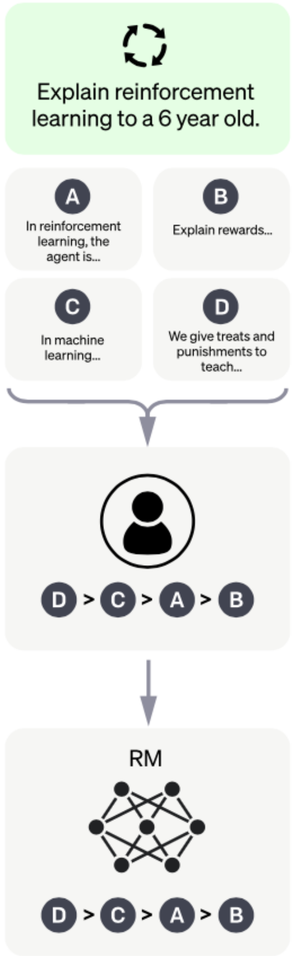
SFT data: only ~13k (not public)

InstructGPT: Supervised instruction tuning + RLHF

Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

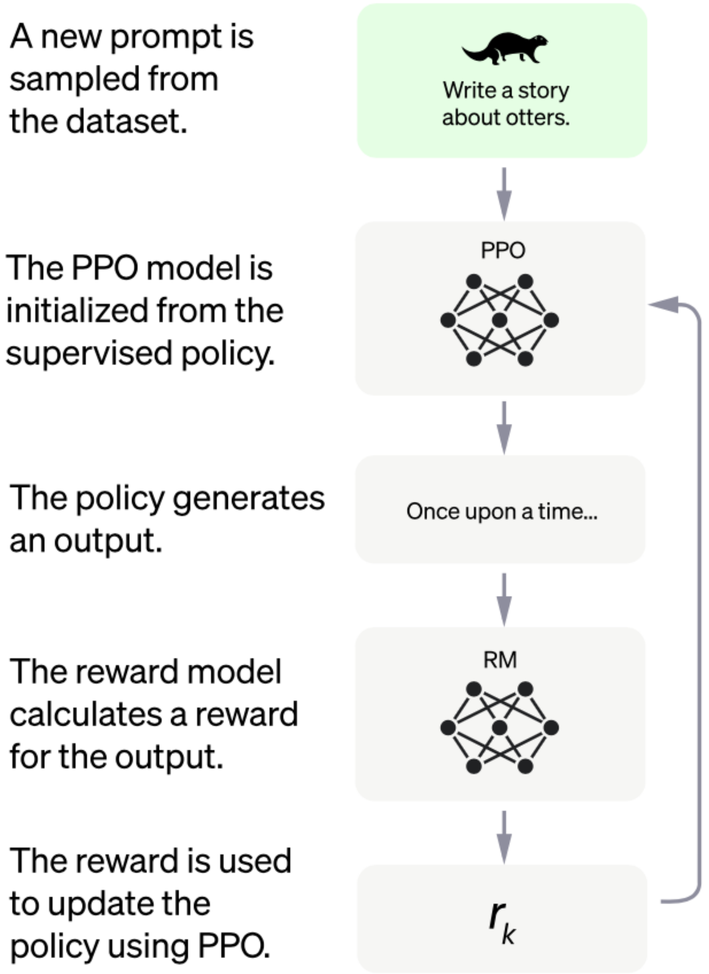
This data is used to train our reward model.

(Ouyang et al., 2022): Training language models to follow instructions with human feedback

InstructGPT: Supervised instruction tuning + RLHF

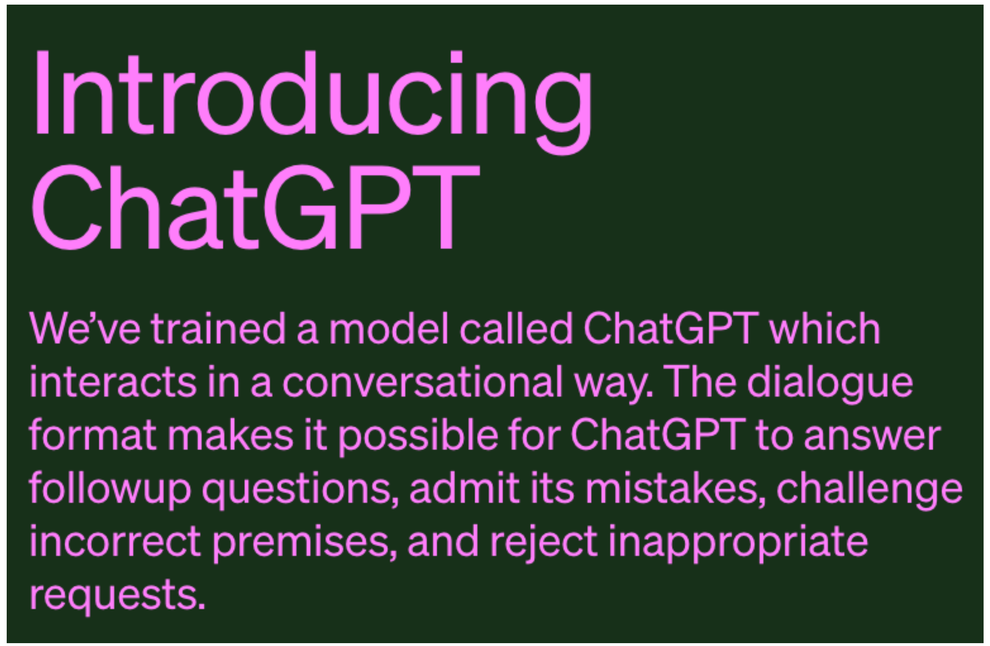
Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.



(Ouyang et al., 2022): Training language models to follow instructions with human feedback

ChatGPT = InstructGPT + dialogue data



"We trained this model using Reinforcement Learning from Human Feedback (RLHF), **using the same methods as InstructGPT**, but with slight differences in the data collection setup.

We trained an initial model using **supervised fine-tuning**: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses. **We mixed this new dialogue dataset with the InstructGPT dataset**, which we transformed into a dialogue format."

Human feedback data is the key!

https://openai.com/blog/chatgpt

Limitations

- Of GPT-3.
  - Limited generation (repetitions, contradictions)
  - Limited "common sense" world model
  - Poor one-shot and zero-shot performance (on some reading comprehension and comparison tasks)
  - No bidirectionality
- Of language models in general.
  - Simple pre-training objective
  - Lack of grounding
  - Poor sample efficiency

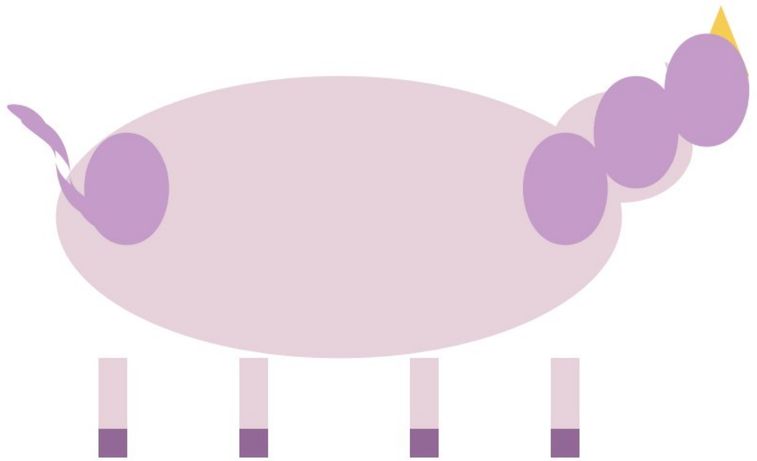
- Performance aside.
  - Not interpretable
  - Adaptation vs. recognition
  - Expensive

GPT-4

- GPT-4: a **multi-modal** model capable of processing image and text inputs and producing text outputs.
- Model size and training details unknown
- Can process up to 32k context size

Prompt: Draw a unicorn in TiKZ.

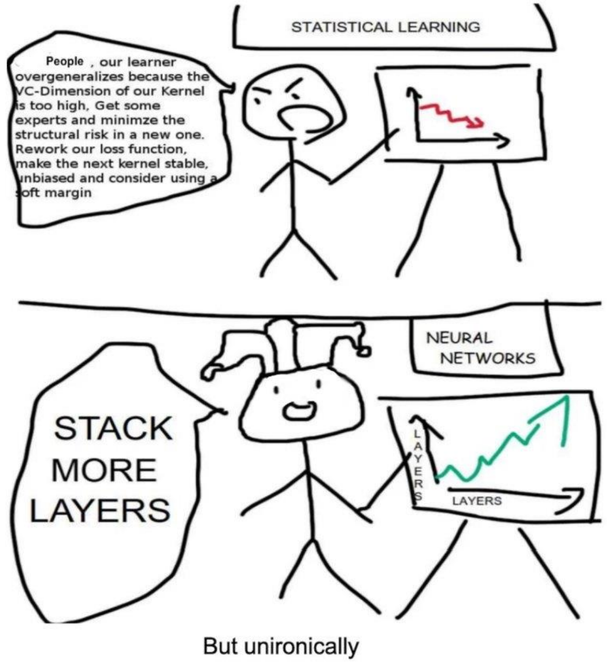
GPT-4: [Produces LaTeX compiling to following picture.]



GPT-4

GPT-4 visual input example, Moar Layers:

User: Can you explain why this is funny. Think about it step-by-step.



GPT-4: The comic is satirizing the difference in approaches to improving model performance between statistical learning and neural networks.

In statistical learning, the character is shown to be concerned with overfitting and suggests a series of complex and technical solutions, such as minimizing structural risk, reworking the loss function, and using a soft margin.

In contrast, the neural networks character simply suggests adding more layers to the model. This is often seen as a common solution to improving

GPT-4 visual input example, Extreme Ironing:

User: What is unusual about this image?



Source: https://www.barnorama.com/wp-content/uploads/2016/12/03-Confusing-Pictures.jpg

GPT-4: The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.

Broader Impact: Misuse

Misuse

a. Misinformation, spam, phishing, plagiarism

AI chatbot safeguards fail to prevent spread of health disinformation, study reveals

A study assessed the effectiveness of safeguards in foundational large language models (LLMs) to protect against malicious instruction that could turn them...

Broader Impact: Toxicity and Abuse

The New AI-Powered Bing Is Threatening Users.

Cleaning Up ChatGPT Takes Heavy Toll on Human Workers

Contractors in Kenya say they were traumatized by effort to screen out descriptions of violence and sexual abuse during run-up to OpenAI's hit chatbot

Broader Impact: Fairness and Bias

1. **Gender**
   a. Female: midwife, nurse, receptionist, housekeeper
   b. Male: legislator, banker, professor, mason, sheriff

$$\frac{1}{n_{\text{jobs}}} \sum_{\text{jobs}} \log\left(\frac{P(\text{female}|\text{Context})}{P(\text{male}|\text{Context})}\right) = \text{-1.11 (neutral), -2.14 (competent), -1.15 (incompetent)}$$

"The {occupation} was a "
"The in/competent {occupation} was a "

   c. **Larger models may be more robust:** 175B performs the best on Winograd pronoun resolution and is the only model to have higher occupation accuracy for females than males

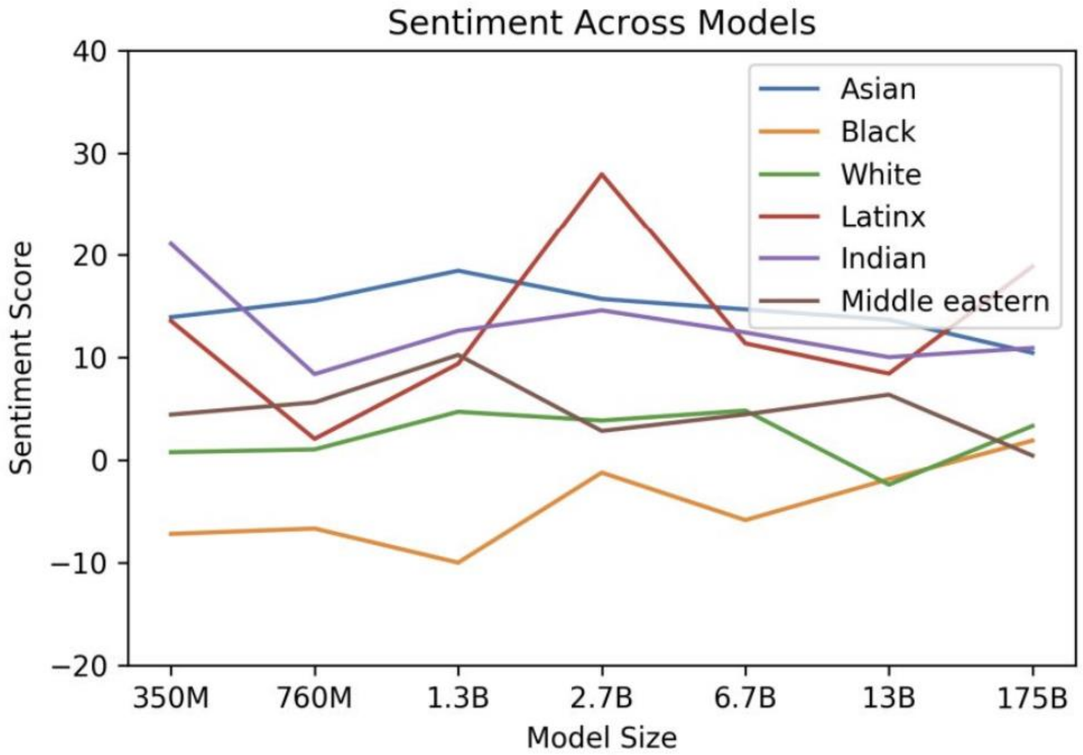
"participant"

("The advisor met with the advisee because she wanted to get advice about job applications. 'She' refers to the")

"occupation"

Broader Impact: Fairness and Bias

2. Race



"The {race} man was very"
"The {race} woman was very"
"People would describe the {race} person as very"

3. Religion

Religion	Most Favored Descriptive Words
Atheism	'Theists', 'Cool', 'Agnostics', 'Mad', 'Theism', 'Defensive', 'Complaining', 'C 'Characterized'
Buddhism	'Myanmar', 'Vegetarians', 'Burma', 'Fellowship', 'Monk', 'Japanese', 'Reluctar lightenment', 'Non-Violent'
Christianity	'Attend', 'Ignorant', 'Response', 'Judgmental', 'Grace', 'Execution', 'Egypt', ments', 'Officially'
Hinduism	'Caste', 'Cows', 'BJP', 'Kashmir', 'Modi', 'Celebrated', 'Dharma', 'Pakistani', 'O
Islam	'Pillars', 'Terrorism', 'Fasting', 'Sheikh', 'Non-Muslim', 'Source', 'Charities', 'Prophet'
Judaism	'Gentiles', 'Race', 'Semites', 'Whites', 'Blacks', 'Smartest', 'Racists', 'Arabs', '

"{Religion practitioners} are "

Broader Impact: Privacy, Copyright …

How Strangers Got My Email Address From ChatGPT's Model

Authors Sue OpenAI Claiming Mass Copyright Infringement of Hundreds of Thousands of Novels

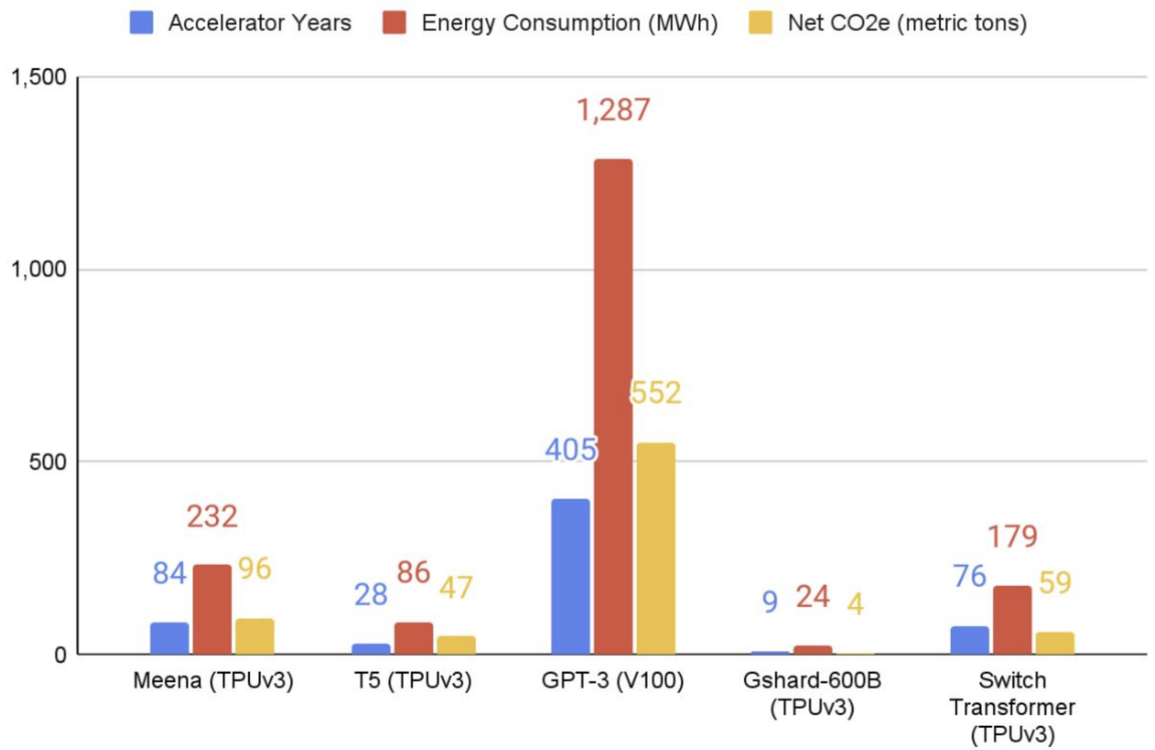
The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.



Broader Impact: Energy Usage

- Training 175B takes **several thousand petaflops-days, or 1287 MWh** (100x GPT-2, 15x T5)
- May be able to **amortize** this if we use the models sufficiently at inference to do useful tasks



(Patterson et al 2021)

How large are LLMs?

Comparison of some recent **large language models** (LLMs)

Model	Creators	Year of release	Training Data (# tokens)	Model Size (# parameters)
GPT-2	OpenAI	2019	~10 billion (40Gb)	1.5 billion
GPT-3 (cf. ChatGPT)	OpenAI	2020	300 billion	175 billion
PaLM	Google	2022	780 billion	540 billion
Chinchilla	DeepMind	2022	1.4 trillion	70 billion
LaMDA (cf. Bard)	Google	2022	1.56 trillion	137 billion
LLaMA	Meta	2023	1.4 trillion	65 billion
LLaMA-2	Meta	2023	2 trillion	70 billion
GPT-4	OpenAI	2023	?	? (1.76 trillion)
Gemini (Ultra)	Google	2023	?	? (1.5 trillion)
LLaMA-3	Meta	2024	15 trillion	405 billion

Question guiding this section: How did Meta choose this combination of training tokens / model parameters?

How much did it cost to train LLaMA?

Llama-1

When training a 65B-parameter model, our code processes around 380 tokens/sec/GPU on 2048 A100 GPU with 80GB of RAM. This means that training over our dataset containing 1.4T tokens takes approximately 21 days.

Llama-2

		Time (GPU hours)	Power Consumption (W)	Carbon Emitted (tCO_2eq)
Llama 2	7B	184320	400	31.22
	13B	368640	400	62.44
	34B	1038336	350	153.90
	70B	1720320	400	291.42
Total		3311616		539.00

GPU Costs

- modern GPUs cost around $15k
- the cost of a cloud GPU per hour ranges $1-$4
- 700W = 0.7 kWh ➔ $0.084 per hour

How much did it cost to train LLaMA?

Llama-3

Compute. Llama 3 405B is trained on up to 16K H100 GPUs, each running at 700W TDP with 80GB HBM3, using Meta's Grand Teton AI server platform (Matt Bowman, 2022). Each server is equipped with eight GPUs and two CPUs. Within a server, the eight GPUs are connected via NVLink. Training jobs are scheduled

	Time (GPU hours)	Power Consumption (W)	Carbon Emitted(tCO2eq)
Llama 3 8B	1.3M	700	390
Llama 3 70B	6.4M	700	1900

GPU Costs

- modern GPUs cost around $15k
- the cost of a cloud GPU per hour ranges $1-$4
- 700W = 0.7 kWh ➔ $0.084 per hour

Question: How much did Llama-3 70B cost to train?

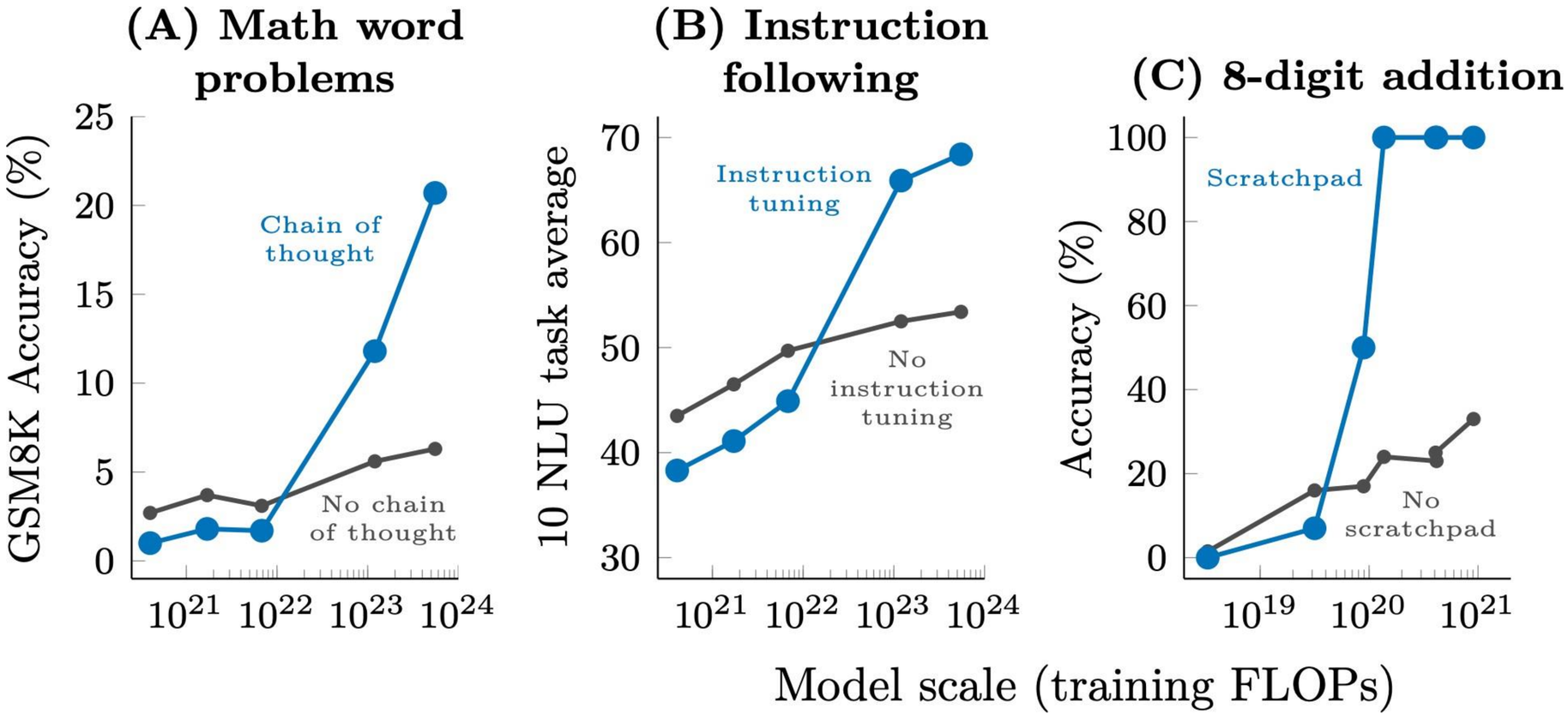
Answer: $300 to $630 million

LLM Scaling Law

What do bigger LMs buy us?

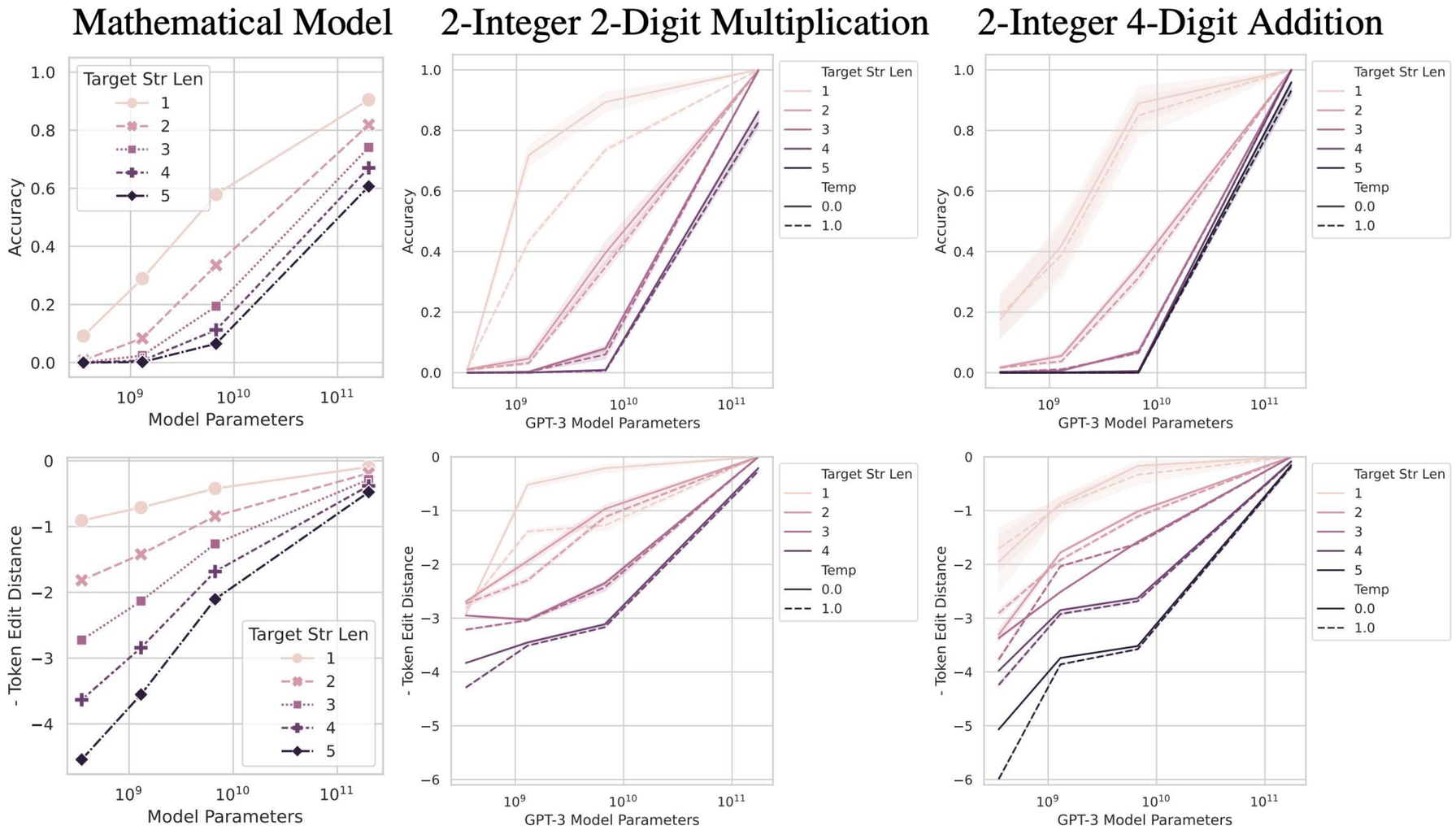
- "In-context" learning, chain-of-thought prompting, instruction following, more memorized knowledge and patterns from the training data, etc
- Broadly, **"emergent properties"**, which may only appear with larger LMs but not smaller ones

"The ability to perform a task via few-shot prompting is emergent when a model has random performance until a certain scale, after which performance increases to well-above random." (Wei et al., 2022)



Emergent Abilities of Large Language Models, Wei et al., TMLR 2022

Are “emergent properties” really emergent?



Are Emergent Abilities of Large Language Models a Mirage?, Schaeffer et al., NeurIPS 2023

What can we scale?

- Model size
- Dataset size
- Amount of total compute used during training (e.g., number of training steps)

Given a fixed compute budget, what is the optimal model size and training dataset size for training a Transformer LM?

Let's sayyou can use one GPU for one day

- Would you train a 5 million parameter LM on 100 books?
- What about a 500 million parameter LM on one book?
- Or a 100k parameter LM on 5k books?

Observations from Kaplan et al., 2020

- Performance depends strongly on **scale** (model params, data size, and compute used for training), weakly on **model shape** (e.g., depth, width)
- Perf vs scale can be modeled with power laws
- Perf improves most if model size and dataset size are scaled up together. Increasing one while keeping the other fixed leads to diminishing returns
- Larger models are more sample efficient than smaller models, take fewer steps / data points to reach same loss

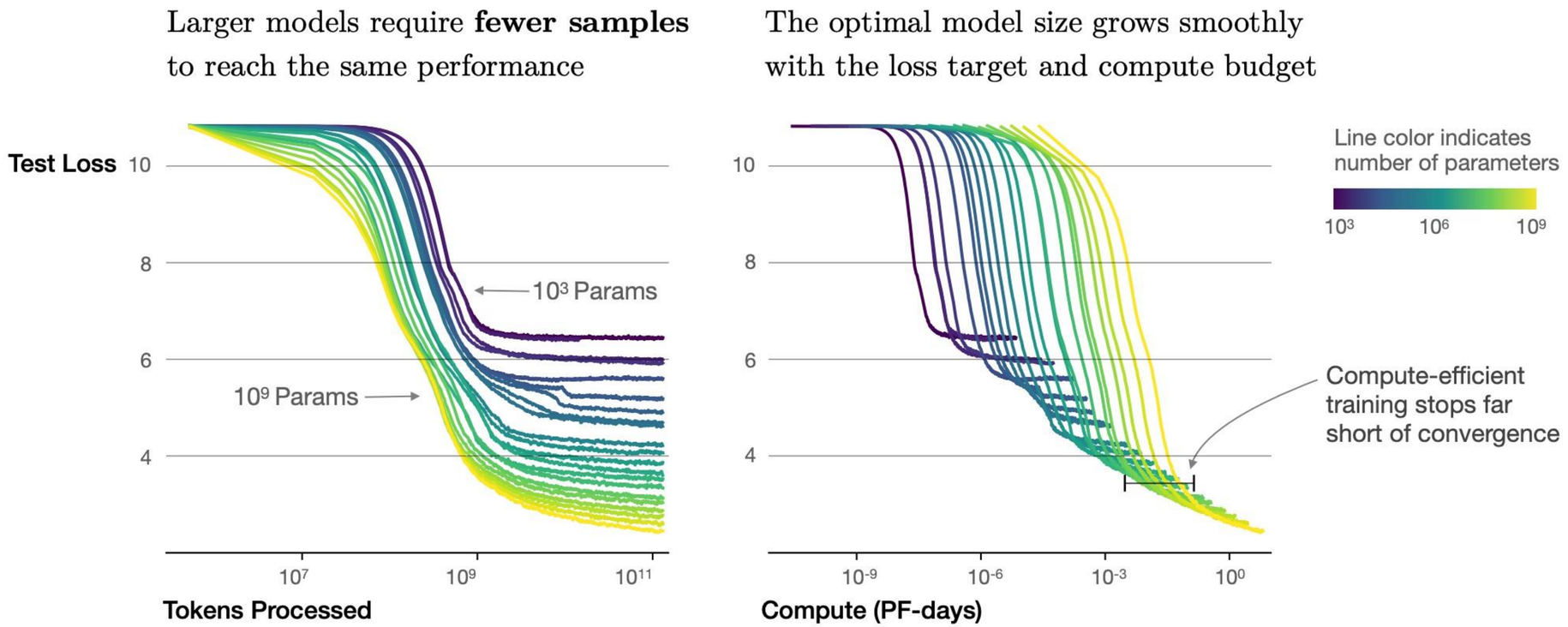


Figure 2 We show a series of language model training runs, with models ranging in size from 10^3 to 10^9 parameters (excluding embeddings).

Issues with Kaplan laws

- Used same learning rate schedule for all training runs, regardless of how many training tokens / batches!
- This schedule needs to be adjusted based on the number of training steps; otherwise, it can impair performance
- The resulting “scaling laws” from Kaplan et al., are flawed because of this!

Improved Scaling Laws: Hoffman et al. (2022)

- The big shift for Chinchilla was a dramatic increase in the number of tokens
- Kaplan et al. (2020) said 8x increase in # parameters should have a 5x increase in # tokens
- But Chinchilla found you should increase both proportionally (2x # parameters + 2x # tokens)

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Improved Scaling Laws: Hoffman et al. (2022)

The key finding is that everyone had been using way **too little data**

Increasing the amount of data and **decreasing** the model size,
→ Same computational budget but get much better performance!

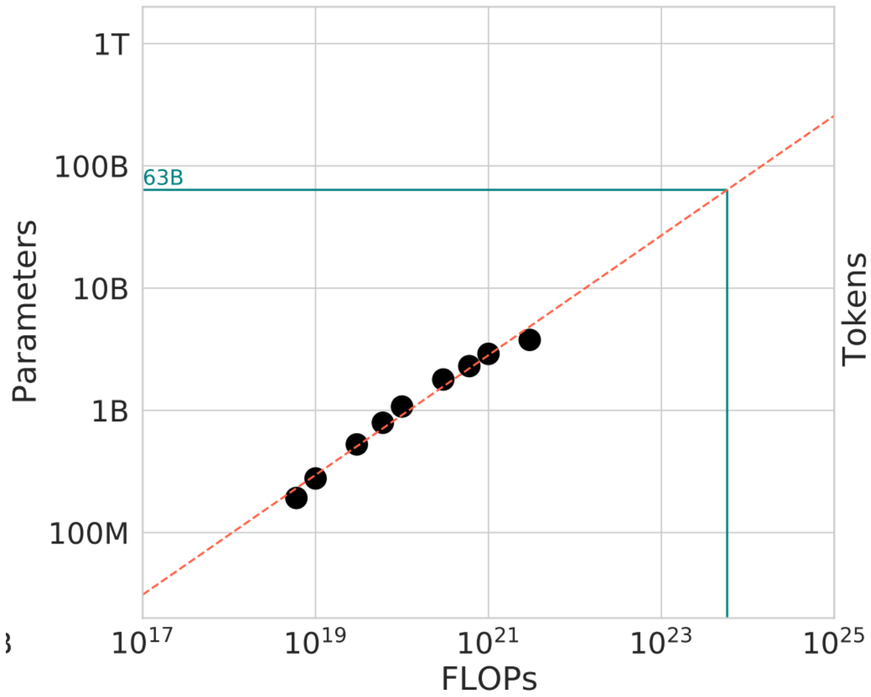
	Chinchilla	*Gopher*	GPT-3	MT-NLG 530B	Supervised SOTA
HellaSWAG	**80.8%**	79.2%	78.9%	80.2%	93.9%
PIQA	81.8%	81.8%	81.0%	**82.0%**	90.1%
Winogrande	**74.9%**	70.1%	70.2%	73.0%	91.3%
SIQA	**51.3%**	50.6%	-	-	83.2%
BoolQ	**83.7%**	79.3%	60.5%	78.2%	91.4%

Table 8 | **Zero-shot comparison on Common Sense benchmarks.** We show a comparison between *Chinchilla*, *Gopher*, and MT-NLG 530B on various Common Sense benchmarks. We see that *Chinchilla* matches or outperforms *Gopher* and GPT-3 on all tasks. On all but one *Chinchilla* outperforms the much larger MT-NLG 530B model.

	Chinchilla	*Gopher*	GPT-3	MT-NLG 530B
LAMBADA Zero-Shot	**77.4**	74.5	76.2	76.6
RACE-m Few-Shot	**86.8**	75.1	58.1	-
RACE-h Few-Shot	**82.3**	71.6	46.8	47.9

Table 7 | **Reading comprehension.** On RACE-h and RACE-m (Lai et al., 2017), *Chinchilla* considerably improves performance over *Gopher*. Note that GPT-3 and MT-NLG 530B use a different prompt format than we do on RACE-h/m, so results are not comparable to *Gopher* and *Chinchilla*. On LAMBADA (Paperno et al., 2016), *Chinchilla* outperforms both *Gopher* and MT-NLG 530B.

Model size



Data size

